

What are Large Language Models?



LLM - 30B + -> text ip -> text op

SLM - Small LM < 1B -> text ip -> text op

VLM - Vision LM - text + image ip -> text

VLA - Vision Language Action Model - Robotics -> text + image →
action

Multi model ip (text, voice, image, video) → op (text, voice, image,
video)



Three Key Components

1

LARGE

Billions of parameters
trained on terabytes of data
(GPT-4: ~1.7T parameters)

2

LANGUAGE

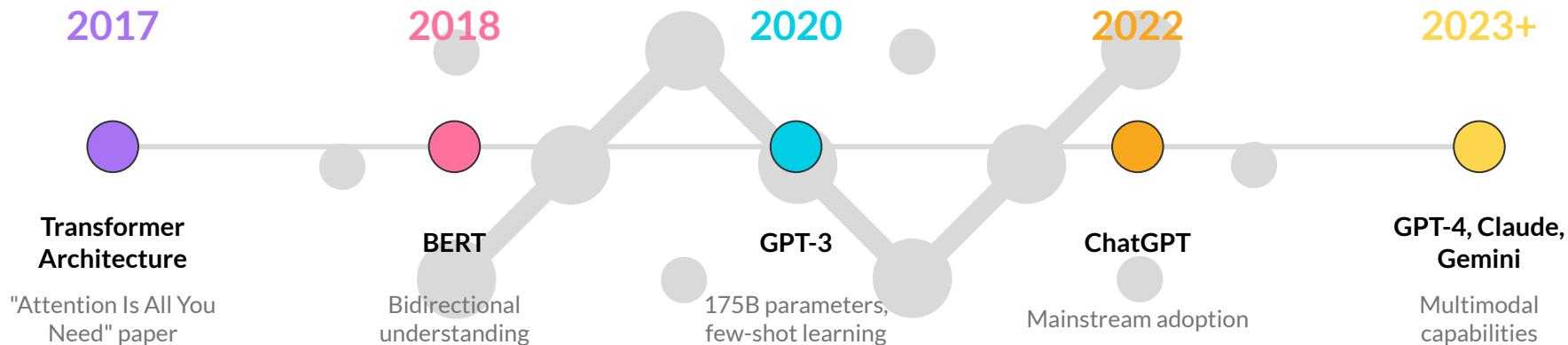
Specialized in processing
text — understands context,
grammar, semantics

3

MODELS

Mathematical
representations predicting
probability distributions
over text

Evolution Timeline

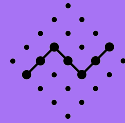


What are LLMs?

Capabilities vs Limitations



✓ Can Do	✗ Cannot Do
Generate text, code, stories	True understanding
Answer questions	Guarantee factual accuracy
Summarize documents	Access real-time data
Translate languages	Consistent outputs
Follow logical reasoning	Reliable math calculations

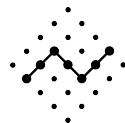


Transformer Architecture

Why Transformers? RNNs vs Transformers



Aspect	RNN	Transformer
Processing	Sequential (word by word)	Parallel (all words at once)
Speed	Slow (can't parallelize)	Fast (massively parallel)
Long-range	Forgets distant context	Direct connections (attention)



Transformer Building Blocks

1

Tokenization & Embeddings

Text → Tokens → Numerical vectors (e.g., 768 dimensions)

2

Positional Encoding

Adds position information (1st, 2nd, 3rd word...)

3

Multi-Head Self-Attention

Lets words focus on relevant context

4

Feed-Forward Networks

Process each token independently after attention

5

Stacking

12-96+ transformer blocks — Early layers: patterns → Deep layers: abstract reasoning

Classic Categorical Encoders



These methods are primarily used for converting categorical features (like colors, sizes, or IDs) into numbers.

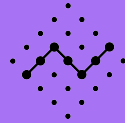
- **Label Encoding:** Assigns a unique integer to each category (e.g., "small" becomes 0, "medium" becomes 1, "large" becomes 2).
 - **Best for:** Ordinal data, where categories have a natural, meaningful order.
- **One-Hot Encoding:** Creates new binary columns for each category, marking the presence of a category with a 1 and absence with a 0.
 - **Best for:** Nominal data, where categories have no inherent order, and when the number of unique categories is small.
- **Ordinal Encoding:** Similar to label encoding, but the user manually specifies the mapping to preserve the correct order.
 - **Best for:** Ordinal data, especially when you need specific integer values that respect the order.
- **Binary Encoding:** Converts categories to binary code and splits the binary digits into separate columns. This reduces dimensionality compared to one-hot encoding for high-cardinality data.
 - **Best for:** High-cardinality nominal data (many unique categories), to avoid the "dimensionality explosion" of one-hot encoding.
- **Target Encoding (Mean Encoding):** Replaces each category with the mean of the target variable for that category.
 - **Best for:** High-cardinality data where the category's relationship to the target variable is important.

Text-Specific Vectorization and Embeddings

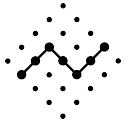
These methods are specifically designed to handle unstructured text data, often used in Natural Language Processing (NLP).

- **Count Vectorization/Bag of Words (BoW):** Creates a dictionary of all unique words and represents each document as a vector where each entry is the count (or frequency) of a specific word.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** This technique, implemented with libraries like `TfidfVectorizer` from `scikit-learn`, assigns numerical values to words based on how frequently they appear in a document relative to the entire corpus.
- **Word Embeddings (Word2Vec, GloVe, BERT, etc.):** These are advanced techniques that map words or phrases into a dense, continuous vector space (an embedding space) where the distance between vectors captures semantic relationships and context.
 - **Best for:** Deep learning models and complex NLP tasks that require understanding the meaning and context of words.
- **Byte Pair Encoding (BPE) / Tokenization:** A subword tokenization method often used in modern NLP that breaks down text into common character sequences (byte pairs) to handle large vocabularies and rare words efficiently.

The choice of encoding method depends on the nature of the data and the specific machine learning algorithm being used.



Attention Mechanism

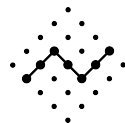


Core Concept

Human Analogy:

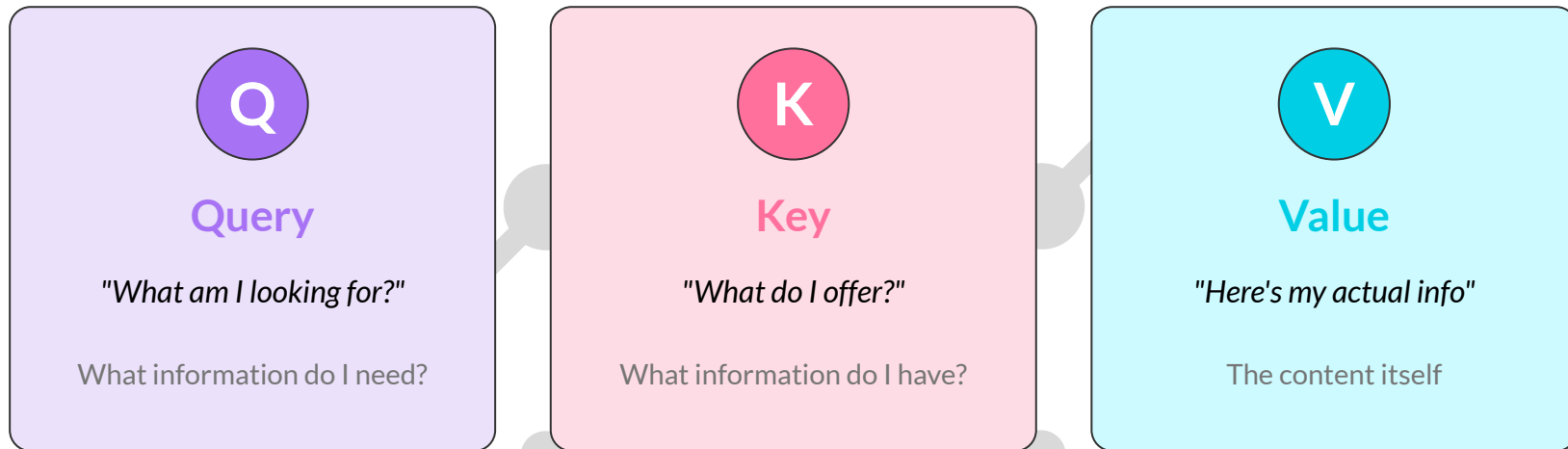
*“The **trophy** doesn’t fit in the **suitcase** because **it** is too **large**.”*

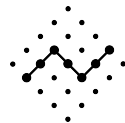
- Your brain knows **"it"** = **"trophy"** (not suitcase)
- You paid attention to **"large"** to figure this out
- This is what **Transformers** do mathematically!



Query-Key-Value Mechanism

Every word creates 3 vectors:





Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

In 4 Steps:

Step 1: Calculate similarity between Query and all Keys

Step 2: Scale the scores (divide by $\sqrt{d_k}$)

Step 3: Softmax → convert to probabilities (sum = 1)

Step 4: Weighted sum of Values

WHY 'softmax'?



1

Converts scores to probabilities

2

Makes all values positive

3

Amplifies differences

4

Enables gradient flow



Multi-Head Attention

Run 8–96 attention mechanisms in parallel, each learning different patterns:

- **Head 1:** Syntax / grammar
- **Head 2:** Semantic meaning
- **Head 3:** Long-range dependencies
- **Heads 4–N:** Other linguistic patterns

Causal Masking (GPT-style)

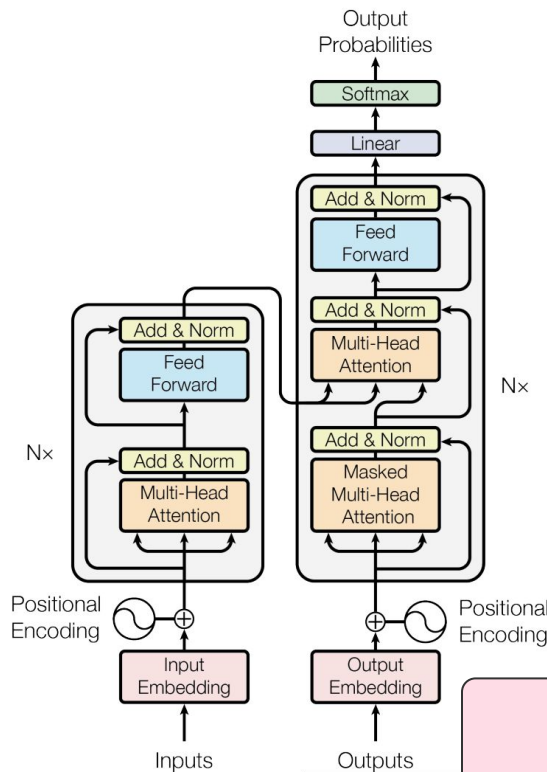
The Rule: When predicting a word, only see previous words (no cheating!)

- Enables left-to-right text generation
- Creates autoregressive property: one token at a time



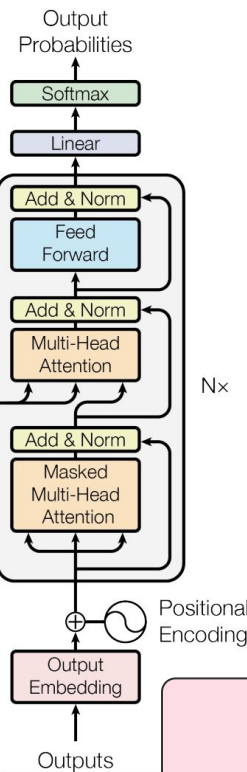
BERT

Encoder



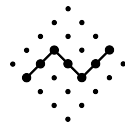
GPT

Decoder

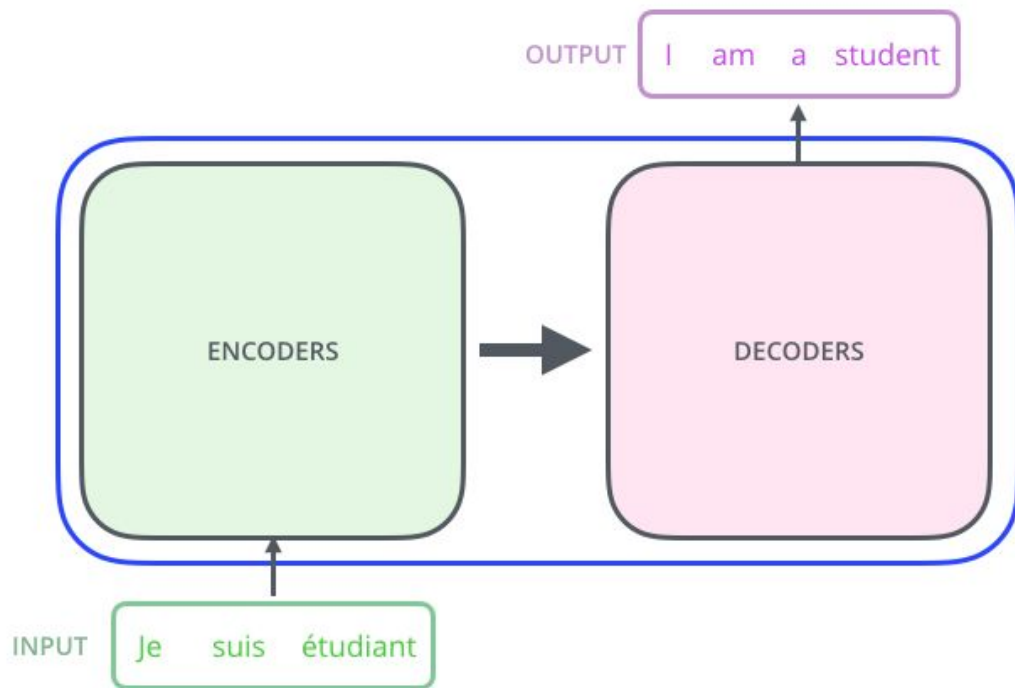


The paper that revolutionized the modern Transformers architecture

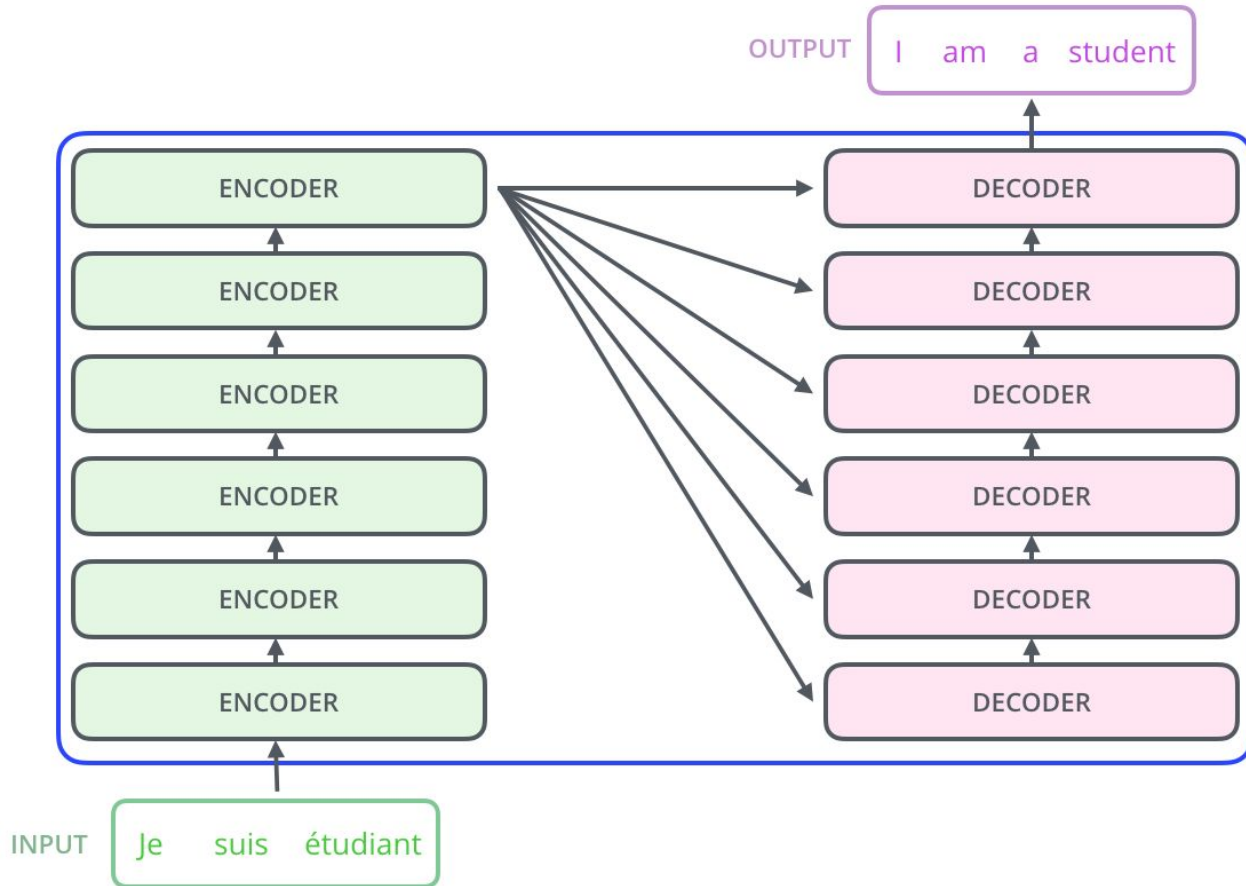
ATTENTION IS ALL YOU NEED



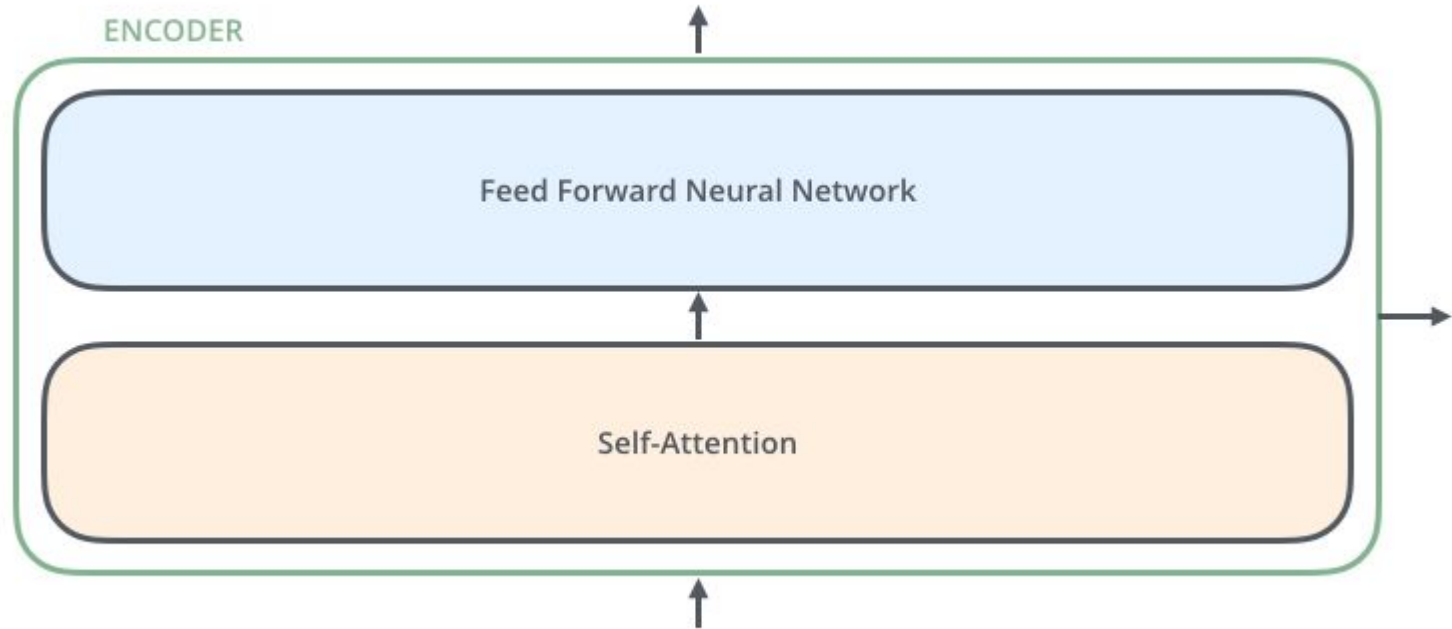
ARCHITECTURE UNDERSTANDING

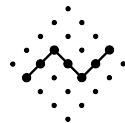


INSIDE ENCODER & DECODER STACK

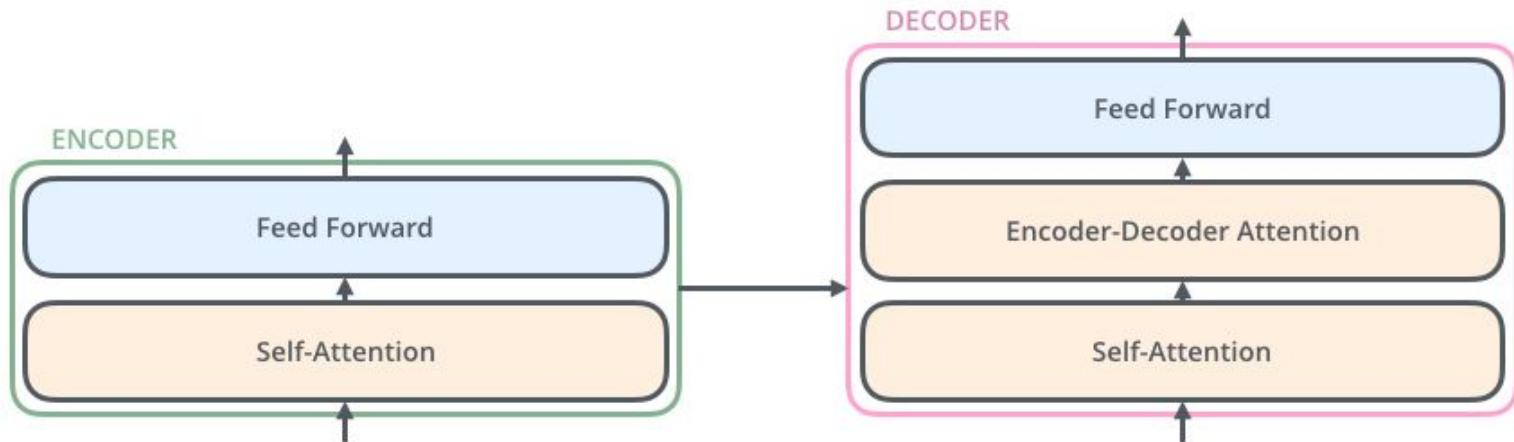


INSIDE SINGLE ENCODER & DECODER

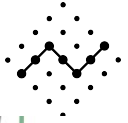




CROSS ATTENTION BLOCK IN DECODER

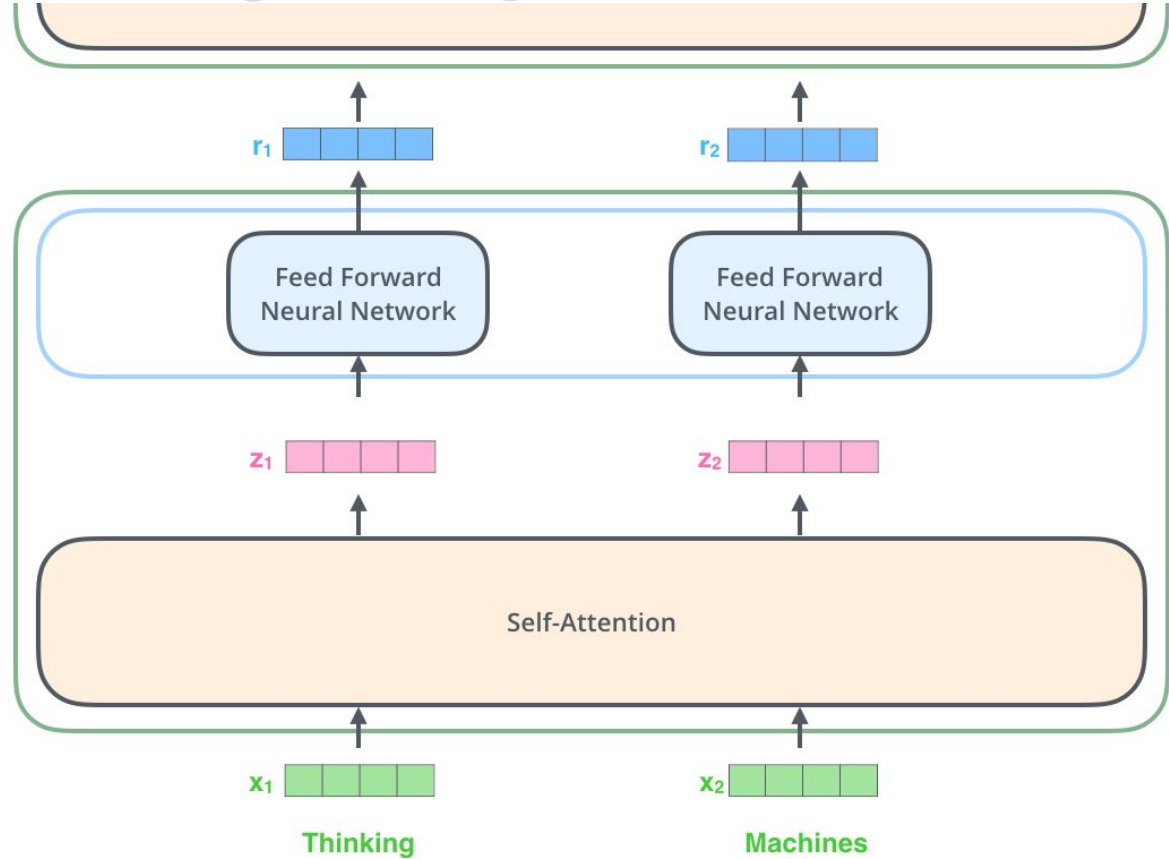


INPUT AND OUTPUT FLOW

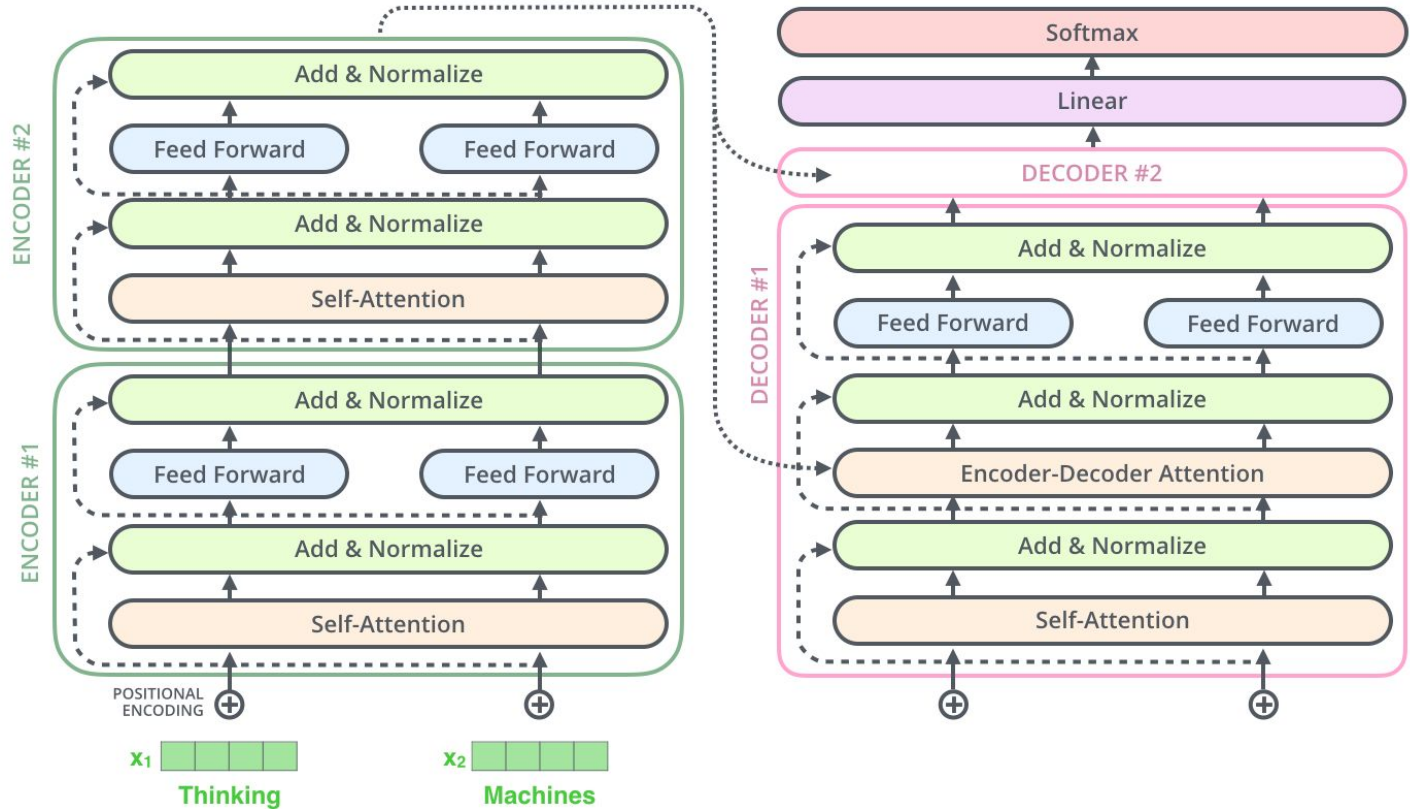
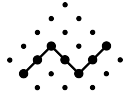


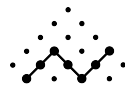
ENCODER #2

ENCODER #1



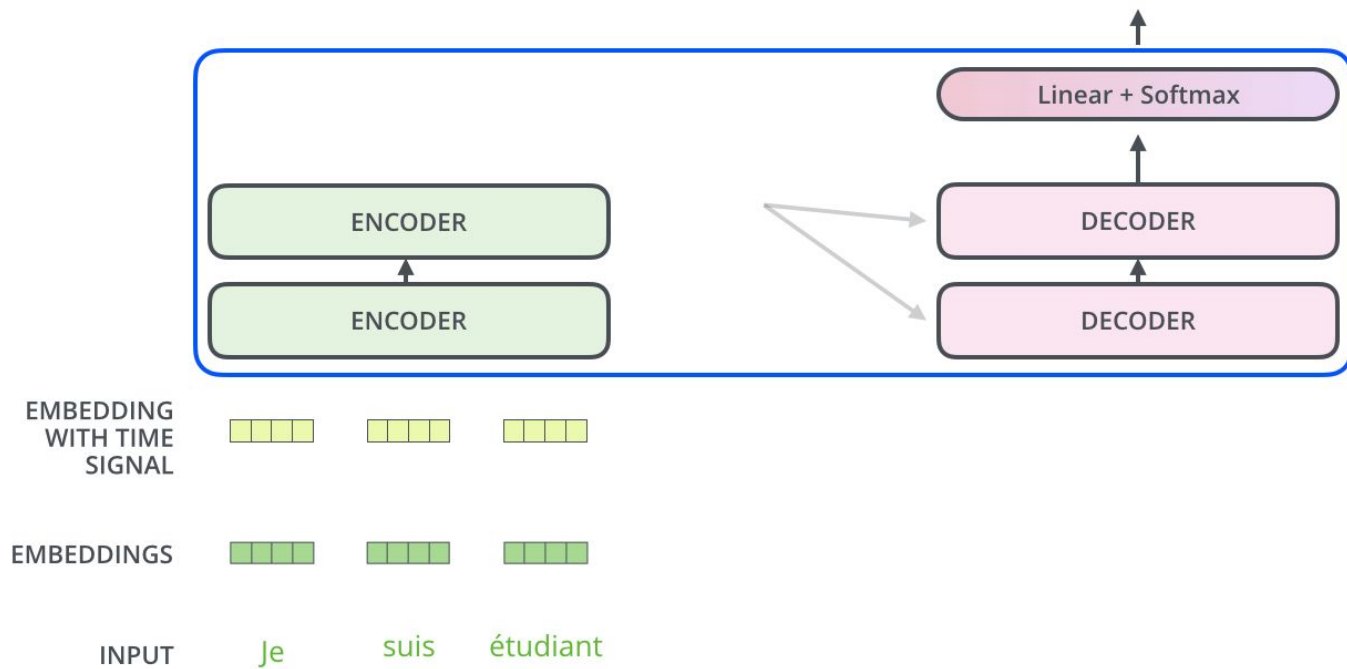
FINAL OVERVIEW





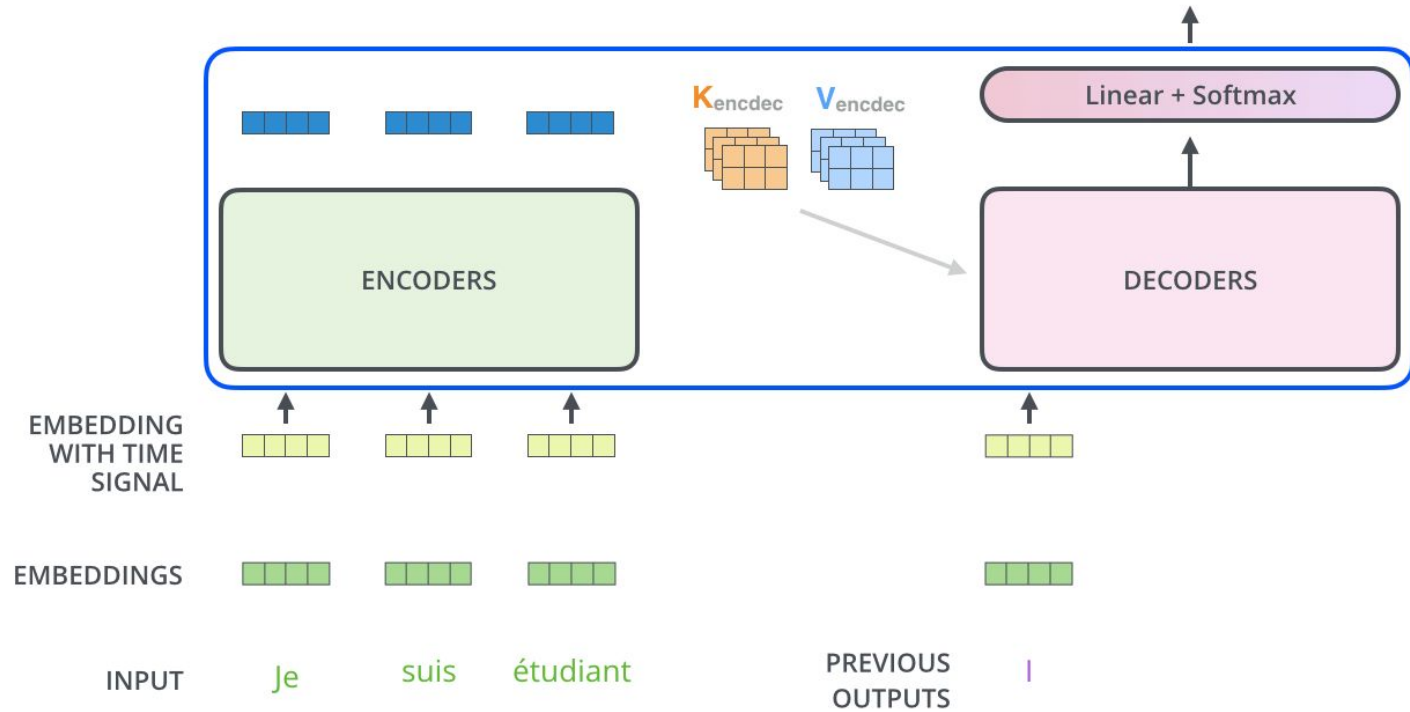
Decoding time step: 1 2 3 4 5 6

OUTPUT



Decoding time step: 1 2 3 4 5 6

OUTPUT |





Types of LLMs



GPT • Claude • Gemini • LLaMA • Mistral

Generative AI & Agentic AI Program



1 GPT (OpenAI)

Provider: OpenAI (Founded 2015, San Francisco)

Model Families

- GPT Series: GPT-3.5, GPT-4, GPT-4 Turbo, GPT-4o, GPT-4.5, GPT-5, GPT-5.3 Codex
- O-Series (Reasoning): o1, o1-mini, o1-pro, o3, o3-mini, o4

Architecture

- Decoder-only Transformer with Mixture of Experts (MoE)
- RLHF alignment, 128K context window

GPT – Key Innovations & Performance



Key Innovations

- GPT-4o: Native multimodal with real-time audio (<200ms latency)
- O-Series: Chain-of-thought reasoning for complex tasks
- Extended thinking for multi-step problem solving

Model	MMLU	HumanEval	MATH	Strength
o1	92.3%	–	94.8%	Reasoning
GPT-4o	88.7%	90.2%	76.6%	Multimodal
o3-mini	–	–	86.7%	Efficient

Standout: o1 achieves 83.3% on AIME 2024 (math olympiad) vs GPT-4o's 13.4%



2 Claude (Anthropic)

Provider: Anthropic (Founded 2021, Safety-focused)

Model Evolution

- Claude 1 → Claude 2 → Claude 3 → Claude 3.5 → Claude 3.7 → Claude 4
- Current: Claude 4.5 Sonnet (flagship), Claude 4.5 Haiku (efficient), Claude Opus 4.6 (reasoning)

Architecture

- Decoder-only Transformer with Constitutional AI (CAI)
- RLAIIF (AI feedback), 200K context window (all models)

Claude – Key Innovations & Performance



Key Innovations

- Constitutional AI: Self-supervised safety alignment
- Computer Use: Control computers via screenshots
- Extended Thinking: Visible reasoning (3.7+)
- 200K context: Excellent recall quality across all models
- SWE-bench Leader: Best real-world coding performance

Model	MMLU	HumanE val	MATH	SWE-bench
Claude 4.5 Sonnet	90.5%	95.2%	82.7%	55.8%
Claude 3.5 Sonnet v2	88.7%	93.7%	78.3%	49.0%
Claude 3.5 Haiku	88.3%	90.2%	–	40.6%

Standout: 95.2% on HumanEval (best code generation score)

3 Gemini (Google DeepMind)



Provider: Google DeepMind (Formed 2023 from Brain + DeepMind merger)

Model Generations

- Gemini 1.0 (Ultra, Pro, Nano)
- Gemini 1.5 (Pro with 1M context, Flash)
- Gemini 2.0 (Flash variants, Pro)
- Gemini 2.5, 3.0 (upcoming)

Architecture

- Multimodal-native Transformer (trained on all modalities together)
- Mixture of Experts (MoE), Up to 1M token context

Gemini – Key Innovations & Performance



Key Innovations

- Multimodal from ground-up: Not separate encoders
- 1M context window: Entire codebases and videos
- Native generation: Image + audio output (2.0)
- Cross-modal reasoning: Text/image/video/audio
- Deep Google ecosystem integration

Model	MMLU	HumanEval	MATH	Context
Gemini 2.0 Flash	86.9%	75.9%	74.4%	–
Gemini 1.5 Pro	85.9%	71.9%	67.7%	1M
Gemini 1.0 Ultra	90.0%	74.4%	94.4%	–

Standout: 1M token context with 99.7% needle-in-haystack accuracy



4 LLaMA (Meta AI)

Provider: Meta AI (Meta Platforms, Open-source leader)

Model Evolution

- LLaMA 1 (7B-65B) → LLaMA 2 (7B-70B) → LLaMA 3 (8B-70B)
- LLaMA 3.1 (8B, 70B, 405B)
- LLaMA 3.2 (1B, 3B text | 11B, 90B vision)
- LLaMA 3.3 (70B distilled)
- LLaMA 4 (MoE: Scout, Maverick, Behemoth)

Architecture

- Decoder-only Transformer (efficiency-optimized)
- Open-source (Apache 2.0), 128K context (3.1+)

LLaMA – Key Innovations & Performance



Key Innovations

- Grouped Query Attention (GQA): Faster inference
- RoPE embeddings: Better position encoding
- 405B model: Largest quality open model
- LLaMA 4 MoE: 17B active / 400B+ total
- 3.3 distillation: 70B with 405B performance

Model	MMLU	HumanE val	MATH	GPQA
LLaMA 3.1 405B	87.3%	89.0%	73.8%	51.1%
LLaMA 3.3 70B	86.0%	88.4%	68.0%	46.7%
LLaMA 3.2 3B	63.4%	59.3%	–	–

Standout: Open-source model competitive with GPT-4 class



5

Mistral (AI)

Provider: Mistral AI (Founded April 2023, European AI leader – France)

Model Portfolio

- Core: Mistral 7B → Small (24B) → Medium → Large (123B → 675B MoE)
- Mixtral: 8x7B, 8x22B (MoE architecture)
- Specialized: Pixtral (vision), Ministral (edge), Codestral (code), Mathstral (math), Magistral (reasoning), Devstral (dev)

Architecture

- Decoder-only Transformer with Sliding Window Attention (SWA)
- Mixture of Experts (MoE), Apache 2.0 license, 128K context

Mistral – Key Innovations & Performance



Key Innovations

- Sparse MoE: Activates only 2/8 experts per token
- Sliding Window: Linear scaling for long context
- 80+ languages: Exceptional multilingual support
- Variable resolution: Pixtral handles any image size
- European GDPR compliance built-in

Model	MMLU	HumanEval	MATH	MMMU
Mistral Large 2	84.0%	92.0%	76.9%	–
Pixtral 124B	–	–	–	69.4%
Codestral 22B	–	81.1%	–	–

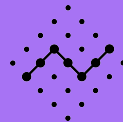
Standout: 92% HumanEval, 80+ language support, European GDPR compliance

Cross-Model Comparison: General Knowledge (MMLU)



Rank	Model	Score	Provider
1	o1	92.3%	OpenAI
2	Claude 4.5 Sonnet	90.5%	Anthropic
3	Gemini 1.0 Ultra	90.0%	Google
4	GPT-4o	88.7%	OpenAI
4	Claude 3.5 Sonnet	88.7%	Anthropic
6	LLaMA 3.1 405B	87.3%	Meta
7	Gemini 2.0 Flash	86.9%	Google
8	LLaMA 3.3 70B	86.0%	Meta
9	Mistral Large 2	84.0%	Mistral

Cross-Model Comparison: Code Generation (HumanEval)



Rank	Model	Score	Provider
1	Claude 4.5 Sonnet	95.2%	Anthropic
2	Claude 3.5 Sonnet	93.7%	Anthropic
3	Mistral Large 2	92.0%	Mistral
4	GPT-4o	90.2%	OpenAI
5	LLaMA 3.1 405B	89.0%	Meta
6	LLaMA 3.3 70B	88.4%	Meta
7	Codestral 22B	81.1%	Mistral
8	Gemini 2.0 Flash	75.9%	Google

Cross-Model Comparison: Mathematics (MATH)



Rank	Model	Score	Provider
1	o1	94.8%	OpenAI
2	Claude 4.5 Sonnet	82.7%	Anthropic
3	Claude 3.5 Sonnet	78.3%	Anthropic
4	Mistral Large 2	76.9%	Mistral
5	GPT-4o	76.6%	OpenAI
6	Gemini 2.0 Flash	74.4%	Google
7	LLaMA 3.1 405B	73.8%	Meta

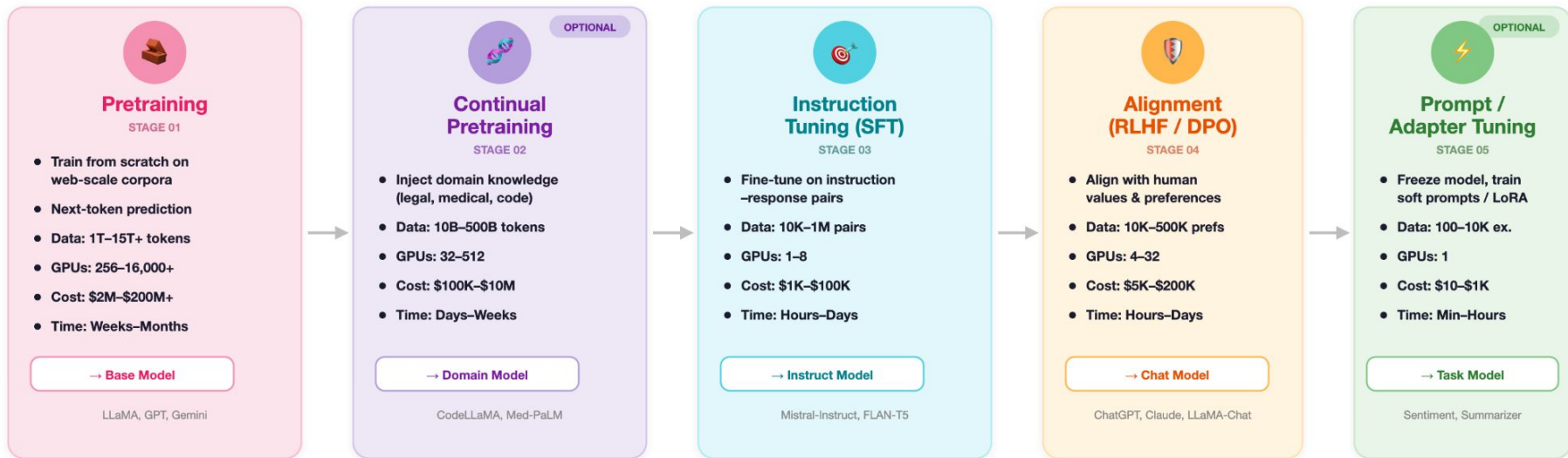
Summary: LLM Comparison



Provider	Flagship	MMLU	Code	Context	Key Strength	Open
OpenAI	o1 / GPT-4o	92.3%	90.2%	128K	Reasoning + Multimodal	✗
Anthropic	Claude 4.5	90.5%	95.2%	200K	Code + Safety	✗
Google	Gemini 2.0	86.9%	75.9%	1M	Context + Multimodal	Partial
Meta	LLaMA 3.3	86.0%	88.4%	128K	Open + Efficient	✓
Mistral	Large 2/3	84–88%	90–92%	128K	Multilingual + MoE	✓



LLM Training Pipeline



Standard path: Stage 1 → 3 → 4 | Stages 2 & 5 are optional | LoRA reduces GPU needs by 4–8x at Stages 3, 4 & 5



Open-Source vs Proprietary Models

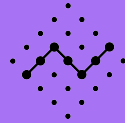


When to Use What

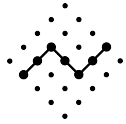
A Comprehensive Guide for AI Practitioners

Nunnari Academy

Generative AI & Agentic AI Program



Understanding the Landscape



1 Proprietary Models

Definition: Commercial LLMs developed and maintained by companies, accessed via paid APIs

O

OpenAI

GPT-4, GPT-4o,
GPT-3.5

A

Anthropic

Claude 3.5 Sonnet,
Claude 3 Opus

G

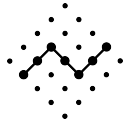
Google

Gemini Pro, Gemini
Ultra

C

Cohere

Command R+



2 Open-Source Models

Definition: Models with publicly available weights that can be downloaded, modified, and deployed independently

M

Meta LLaMA

2, 3, 3.1, 3.2

M

Mistral

7B, 8x7B Mixtral,
8x22B

F

Falcon

7B, 40B, 180B

P

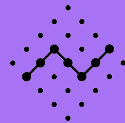
Phi

Microsoft's small
LMs

Q

Qwen

Alibaba's
multilingual

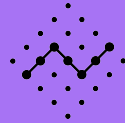


Key Differences

Key Differences Comparison



Factor	Proprietary Models	Open-Source Models
Access	API-based, subscription	Download and self-host
Cost Structure	Pay-per-token	Infrastructure + compute costs
Customization	Limited (prompt eng, fine-tuning APIs)	Full control (modify architecture)
Data Privacy	Data sent to external servers	Data stays on your infrastructure
Performance	Generally state-of-the-art	Rapidly improving, some match
Updates	Automatic, managed by provider	Manual updates required
Scaling	Handled by provider	You manage infrastructure
Compliance	Depends on provider certs	Full control over compliance



When to Use Proprietary Models

Best Use Cases for Proprietary Models



Rapid Prototyping

- Quick API integration
- No infra setup required
- Fast time-to-market



High-Stakes Apps

- Customer-facing chatbots
- Complex reasoning tasks
- Multi-modal applications



Limited Team

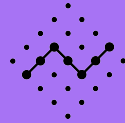
- Small teams without ML eng
- No DevOps capacity
- Focus on app logic



Variable Workloads

- Spiky traffic patterns
- Pay only for what you use
- No idle infrastructure

Cost tip: Usually beneficial when processing < 1–5M tokens/month. Example: 1M GPT-4 tokens ≈ ₹3,000–4,000



When to Use Open-Source Models

Best Use Cases for Open-Source Models



Data Privacy

- Healthcare (patient data)
- Financial services
- Government contracts
- Proprietary business data



High Volume

- Millions of documents
- Continuous background tasks
- Stable traffic patterns
- Break-even: >10M tokens/mo



Domain-Specific

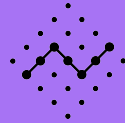
- Industry jargon (legal, medical)
- Regional language support
- Specialized optimization
- Brand voice consistency



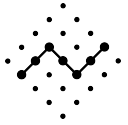
Offline & Edge

- On-premise installations
- Air-gapped environments
- Mobile / edge devices
- Limited connectivity areas

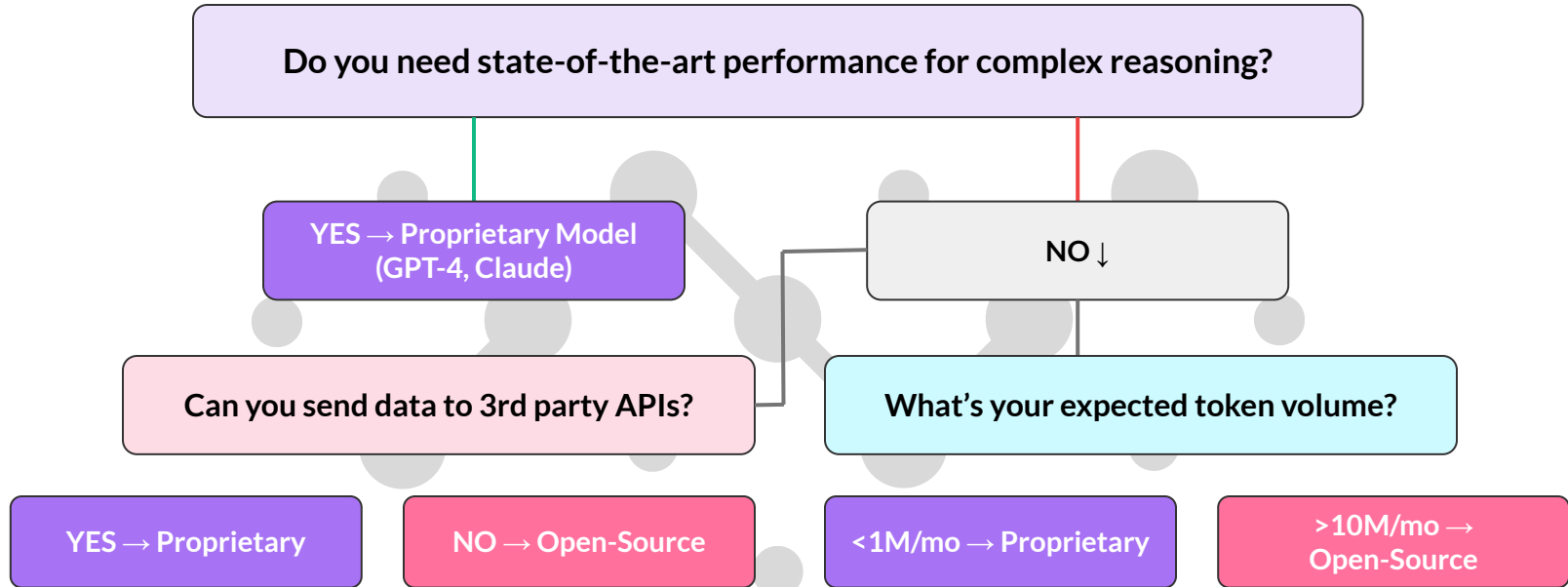
Cost tip: Initial GPU infra ₹50K-5L+ | Cloud compute ₹15K-1L/mo | Typically profitable at >10M tokens/month sustained



Decision Framework



Step 1: Evaluate Your Requirements



Step 2: Consider Hybrid Approaches



1

Smart Routing

- Use GPT-4 for complex queries
- Use open-source 7B for simple FAQs
- Route based on query complexity

Cost savings: 60–70%

2

Dev → Production

- Prototype with proprietary (fast iteration)
- Fine-tune open-source for production
- Optimize cost at scale

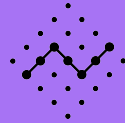
Best of both worlds

3

Tiered Service

- Premium users get GPT-4 / Claude
- Standard users get fine-tuned OS models
- Match cost to user value

Revenue-aligned costs



Key Takeaways

Key Takeaways



No one-size-fits-all — Choose based on your specific use case, requirements, and constraints



Start simple — Use proprietary models for MVP and rapid prototyping



Scale smart — Transition to open-source for high-volume production workloads



Hybrid wins — Combine both strategically for optimal cost and performance



Keep learning — The LLM landscape evolves rapidly, stay updated

The best model meets your requirements for performance, cost, privacy, and deployment constraints.



Tokenization & Context Windows



Understanding How AI Models Process Text

Module 1, Week 1

Nunnari Academy

Generative AI & Agentic AI Program



What is Tokenization?

Breaking down text into smaller units called **tokens** — the basic building blocks AI models understand.

Tokens are not always complete words:



A full word

"hello" → 1
token



Part of a word

"un" + "happy" →
2 tokens



A single character

"!" → 1 token



**Punctuation &
numbers**

"42" , "." , " "



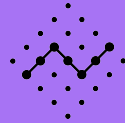
Tokenization Examples

"cat" → = 1 token *Simple, common word*

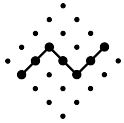
"tokenization"
n" → = 2 tokens *Split into sub-words*

"AI is great!" → = 4 tokens *Includes punctuation*

Key Rule: 1 token $\approx$ 0.75 words (3–4 characters in English) | Common words = fewer tokens | Rare words = more tokens



Context Windows



What is a Context Window?

The model's **working memory** — the maximum amount of text it can process at once.



Working Memory

Everything the model can “see” at one time during processing



Measured in Tokens

Not words — token count determines how much fits in the window



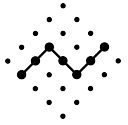
Input + Output

The window includes both what you send AND what the model generates



Has a Limit

When full, older content may be truncated or lost



Context Window Size Examples

200,000

tokens $\approx$ 150,000 words $\approx$ a 500-page book

Early Models

4K tokens

GPT-3.5

16K tokens

GPT-4

128K tokens

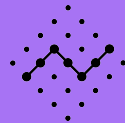
Claude 3.5

200K tokens

Gemini 1.5

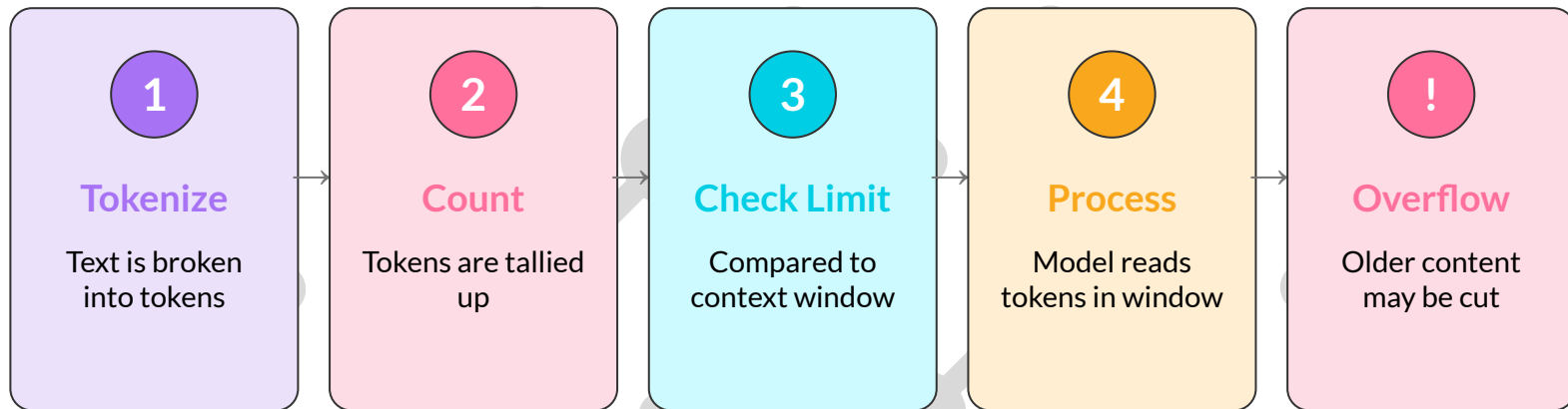
1M tokens

Trend: Larger context windows = more context retained = better understanding



How They Work Together

How They Work Together



Example Flow:

"Hello, how are you today?" → [Hello] [,] [how] [are] [you] [today] [?] → 7 tokens → Fits in window ✓

Why This Matters



1 Determines how much the model can “remember” in a conversation

2 Affects conversation length and document analysis capability

3 Enables processing of longer documents, codebases, and books

4 Impacts quality of responses and context retention over time

Practical Implications

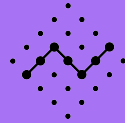


Benefits of Larger Context

- Better understanding of long conversations
- Can analyze entire books, reports, or codebases
- Improved context retention across sessions
- More comprehensive and accurate responses

Trade-offs to Consider

- More tokens = more computational resources
- Higher cost per request for larger windows
- Different models offer different window sizes
- Choose based on your specific use case needs



Key Takeaways

Key Takeaways



Tokenization = breaking text into processable units



Context window = model's memory limit



1 token $\approx$ 0.75 words on average



Larger windows enable more comprehensive AI interactions



Together they define AI model capabilities



Understanding Model Parameters

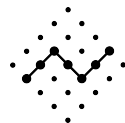


And Their Implications for AI Applications

A Deep Dive into Weights, Scale & Optimization

Nunnari Academy

Generative AI & Agentic AI Program



What Are Model Parameters?

Parameters are the learnable **weights and biases** within a neural network that determine how the model processes information. In LLMs, these are numerical values adjusted during training to capture **language patterns, knowledge, and reasoning**.

The "B" Factor: 1B = 1 billion parameters. A 7B model contains 7 billion trainable weights.

Small

7B – 13B

LLaMA-2 7B, Mistral 7B

Medium

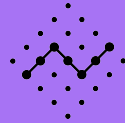
30B – 70B

LLaMA-2 70B

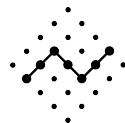
Large

175B+

GPT-3.5, GPT-4



Capability vs. Resource Trade-off



Capability vs. Resource Trade-off

Parameter Size	Capabilities	Hardware	Speed	Cost
7B – 13B	Good for focused tasks	16GB+ GPU	Fast	Low
30B – 70B	Strong reasoning	40GB+ GPU	Moderate	Medium
175B+	Best performance	Multiple GPUs / API	Slow	High



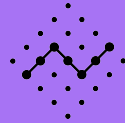
Performance gains diminish with scale — bigger isn't always better



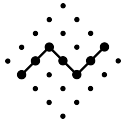
Choice of model size is a strategic decision based on use case



A fine-tuned 7B often outperforms a generic 70B for domain tasks

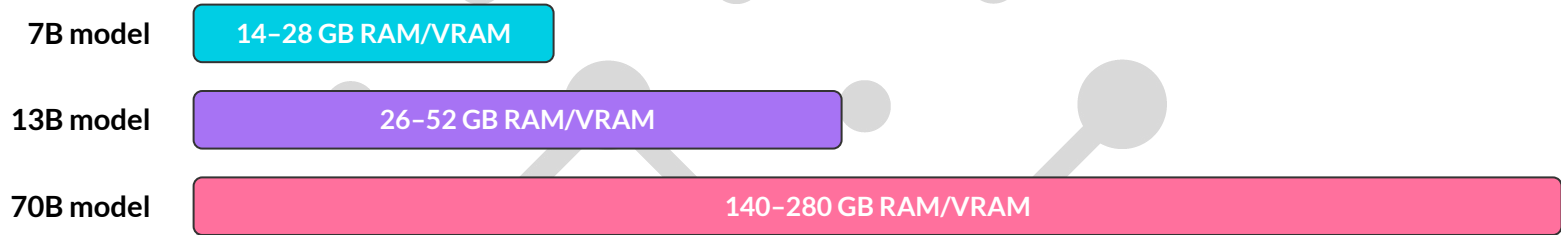


Parameter Implications



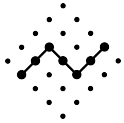
Resource Requirements: Memory

Inference (Running the Model)



Training / Fine-tuning

- Requires 3-4x more memory than inference
- Large models may need multiple GPUs or distributed training
- Gradient accumulation & checkpointing reduce memory footprint



Cost, Speed & Specialization

1

Cost Implications

- API costs scale linearly with parameter count
- 70B model costs ~10x more than 7B
- Self-hosting larger models requires expensive GPU hardware
- Larger models have significantly higher energy costs

2

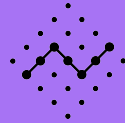
Latency & Speed

- Smaller models: sub-second, ideal for real-time
- Larger models: several seconds, better for batch
- Smaller models handle more concurrent requests

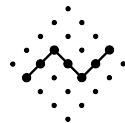
3

Specialization

- Large models: better generalists across diverse tasks
- Small models: excel when fine-tuned for specific domains
- Fine-tuned 7B often outperforms generic 70B

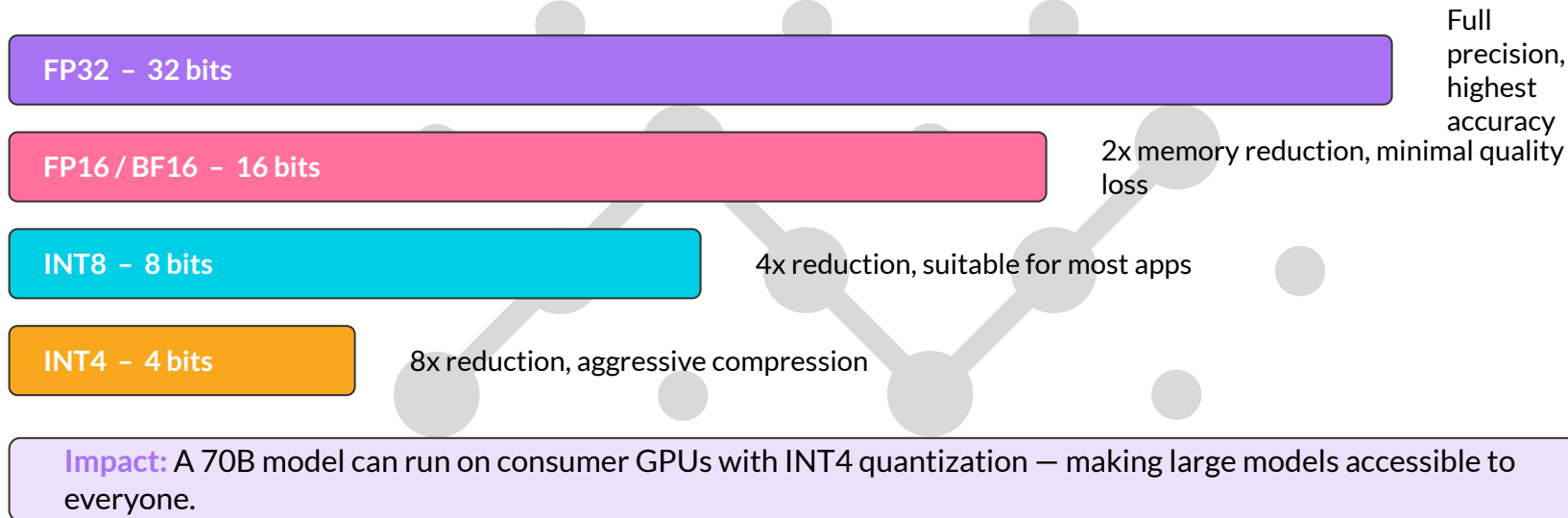


Optimization Techniques

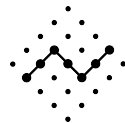


1 Quantization

Reducing numerical precision to decrease memory and increase speed



Distillation & Mixture of Experts



2

Model Distillation

Training smaller 'student' models to mimic larger 'teacher' models

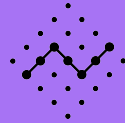
- Large model generates training data for smaller model
- Retain 95%+ of teacher performance with 10x fewer params
- Example: DistilBERT (66M) matches BERT-base (110M)
- Deploy compact models for edge & mobile

3

Mixture of Experts (MoE)

Activating only relevant parameters per request

- Multiple specialized 'expert' networks with routing
- Only 10–20% of parameters active per inference
- Example: Mixtral 8x7B (47B total, 13B active)
- Large model capability at small model cost



Best Practices & Recommendations

Best Practices & Recommendations



Golden Rule: Start small, scale up only when necessary. Use the smallest model that meets your quality requirements.

1

Production Strategy

7B–13B models fine-tuned for specific domains often outperform larger generic models while being 10x more cost-effective.

2

Optimization Strategy

Combine quantization with fine-tuning to achieve optimal balance between performance, cost, and latency.

3

Scaling Strategy

Prototype with small models, validate with benchmarks, and only scale to larger models when smaller ones can't meet requirements.

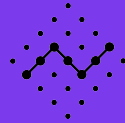


TOKEN ECONOMICS & COST OPTIMIZATION STRATEGIES



Nunnari Academy

Module 1: Foundations & LLM Essentials | Week 1 | Section 1.3



What Are Tokens?

Understanding how LLMs process text



How Tokenization Works

LLMs break text into smaller units called tokens — a token can be a full word, a part of a word, punctuation, or even a space.

Input Text	Tokens	Count
Hello	[Hello]	1
Hello, world!	[Hello] [,] [world] [!]	4
Artificial Intelligence	[Art] [ificial] [Intelligence]	3
supercalifragilistic	[super] [cal] [ifr] [agil] [istic]	5



Rule of Thumb

English: 1 token $\approx$ 4 characters $\approx$ 0.75 words

1,000 words $\approx$ 1,333 tokens

Non-Latin scripts (Tamil, Hindi) use 2–4x more tokens per word



Input vs Output Tokens & Context Windows

Token Types

Input Tokens (Prompt):

System prompt, user message, conversation history, RAG context, function definitions

Output Tokens (Completion):

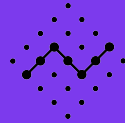
Model response, function calls, JSON output

Output tokens cost 3–5x more than input tokens

Context Windows

Model	Context	Pages
GPT-4o	128K	~200
Claude 3.5 Sonnet	200K	~300
Gemini 1.5 Pro	1M	~1,500
LLaMA 3.1 (8B)	128K	~200

Larger context windows allow more information per request, but also mean higher costs per call.



How LLM Pricing Works

Understanding the cost structure behind API calls





The Pricing Model

Most providers charge per token processed, expressed as cost per million tokens.

$$\text{Total Cost} = (\text{Input Tokens} \times \text{Input Price}) + (\text{Output Tokens} \times \text{Output Price})$$

Model	Input / 1M	Output / 1M	Tier
GPT-4o	\$2.50	\$10.00	Mid-range
GPT-4o-mini	\$0.15	\$0.60	Budget
Claude 3.5 Sonnet	\$3.00	\$15.00	Mid-range
Claude 3.5 Haiku	\$0.80	\$4.00	Budget
Gemini 1.5 Flash	\$0.075	\$0.30	Budget
Gemini 1.5 Pro	\$1.25	\$5.00	Mid-range
LLaMA 3.1 (Self-hosted)	Infra only	Infra only	Self-managed



Cost Calculation: RAG Chatbot Example

Per-Request Breakdown

System prompt:

~500 tokens

RAG context:

~2,000 tokens

User query:

~50 tokens

History (5 turns):

~1,000 tokens

Total Input: 3,550 | Output: 300

GPT-4o

\$0.012 / request

\$3,562/mo

≈ ₹2.97L/mo

GPT-4o-mini

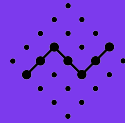
\$0.0007 / request

\$214/mo

≈ ₹17.8K/mo

16x cost reduction by switching models

At 10,000 queries/day. The question: is the quality difference worth ₹2.8 lakhs/month?



The Hidden Costs

Token pricing is just part of the picture



Beyond Token Pricing



Embedding Costs

OpenAI small: \$0.02/1M tokens
OpenAI large: \$0.13/1M tokens
Cohere embed-v3: \$0.10/1M
Local (Sentence Transformers): Free



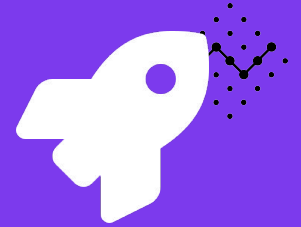
Vector DB Costs

Pinecone: Free → ~\$70/mo
Weaviate Cloud: From ~\$25/mo
Chroma/Qdrant (self-hosted):
Free software + server costs



Infrastructure

GPU instances: \$1–10/hour
API gateway & load balancer
Logging & monitoring tools
Data storage & bandwidth



Cost Optimization Strategies

Where practical engineering meets financial planning



Highest Impact

1 Model Tiering

Simple

FAQs, greetings, status checks

GPT-4o-mini / Gemini Flash / Haiku

Medium

Summarization, basic analysis

GPT-4o / Claude Sonnet

Complex

Multi-step reasoning, code gen, legal

GPT-4 / Claude Opus



50–70% cost reduction. In most apps, 70–80% of queries are simple enough for a budget model.

2

Prompt Optimization



Key Techniques

Trim system prompts

Remove verbose instructions (2,000 → 500 tokens)

Compress RAG context

Summarize retrieved docs before injection

Limit conversation history

Rolling summary + last 2–3 turns

Set max_tokens

Cap output length to prevent waste

Use structured outputs

JSON is shorter than prose

Before vs After

	Before	After
System prompt	2,000	500
RAG context	3,000	1,200
History	2,000	600
Total Input	7,050	2,350
Cost/request	\$0.0176	\$0.0059

66% reduction in input costs

3 Caching



Exact Match Cache

Store responses for identical queries. Simple and effective for FAQs.

Semantic Cache

Use embeddings to find similar past queries. Return cached if similarity > threshold.

Prompt Cache

Providers cache repeated prompt prefixes at reduced rates. Anthropic: 90% discount on cached portion.

Anthropic Prompt Caching

First request:

Cache write at 1.25x price

Subsequent requests:

Cache read at 0.1x price

90% discount
on cached tokens



4 Batching

OpenAI Batch API:

50% cost reduction for non-real-time workloads. Results within 24 hours.

Internal Batching:

Group similar requests into a single prompt with structured output.

Best for:

- Document processing pipelines
- Bulk classification
- Data enrichment
- Nightly report generation

5 Self-Hosted Models

Use API When	Self-Host When
< 100K req/month	> 500K req/month
Need latest models	Simpler tasks (7B–13B)
No GPU team	MLOps capabilities
Variable load	Steady traffic
Quick iteration	Data privacy critical

1M req/mo: **GPT-4o** ₹9.9L | **4o-mini** ₹59K | **Self-hosted 8B**
₹41.7K



6

Evaluation-Driven Optimization

You can't optimize what you can't measure.



Track cost per conversation, not just per API call



Measure quality metrics (accuracy, relevance, faithfulness) alongside cost



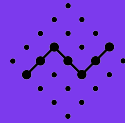
A/B test cheaper models against expensive ones on real traffic



Build a cost-quality Pareto frontier for each task



The goal is not the cheapest model — it's the best quality per rupee spent.



Cost Monitoring & Budgeting

Setting up controls, tracking, and budget safeguards





Monitoring, Budget Controls & Tools

What to Track

- Total tokens per API call (input + output)
- Cost per request, user, conversation
- Daily/weekly/monthly spend trends
- Breakdown by model, feature, user segment

Budget Controls

Hard limits:

Max monthly spend per API key

Rate limiting:

Cap requests per user/hour/day

Alerting:

Notifications at 50%, 80%, 90% of budget

Circuit breakers:

Auto-switch to cheaper models at threshold

Monitoring Tools

Tool	Type	Tracks
LangSmith	Commercial	Traces, latency, cost per chain
Langfuse	Open Source	Token tracking, cost analytics
OpenAI Dashboard	Built-in	Usage, cost per model, limits
Custom Logging	DIY	Full control, custom analytics





Decision Framework

Use this framework for any LLM model and architecture decision:

1

Define Quality Requirements

What accuracy/quality level does this task need?

2

Estimate Volume

How many requests per day/month?

3

Calculate Baseline Cost

What will the best model cost at that volume?

4

Test Cheaper Alternatives

Run evaluations on smaller/cheaper models

5

Apply Optimization Strategies

Caching, prompt trimming, model tiering

6

Monitor and Iterate

Track cost-quality metrics in production



Key Takeaways

- ✓ Tokens are the currency of LLMs — understand how text maps to tokens, especially for non-English languages
- ✓ Output tokens cost 3–5x more than input tokens — controlling output length is high-leverage
- ✓ Model tiering is the single biggest cost lever — route 70%+ of traffic to budget models
- ✓ Prompt optimization compounds — saving 1,000 tokens/request × 10K daily = ₹2.5L/month savings
- ✓ Caching eliminates redundant spend — especially for FAQ-heavy applications
- ✓ Self-hosting is viable for high-volume, simpler tasks — not a replacement for frontier models
- ✓ Always measure before optimizing — set up cost tracking from day one
- ✓ Optimize for quality per rupee, not just lowest cost



Prompt Engineering Fundamentals

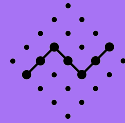


Week 2: Mastering the Art of AI Communication

From Anatomy of Prompts to Role-Based Architecture

Nunnari Academy

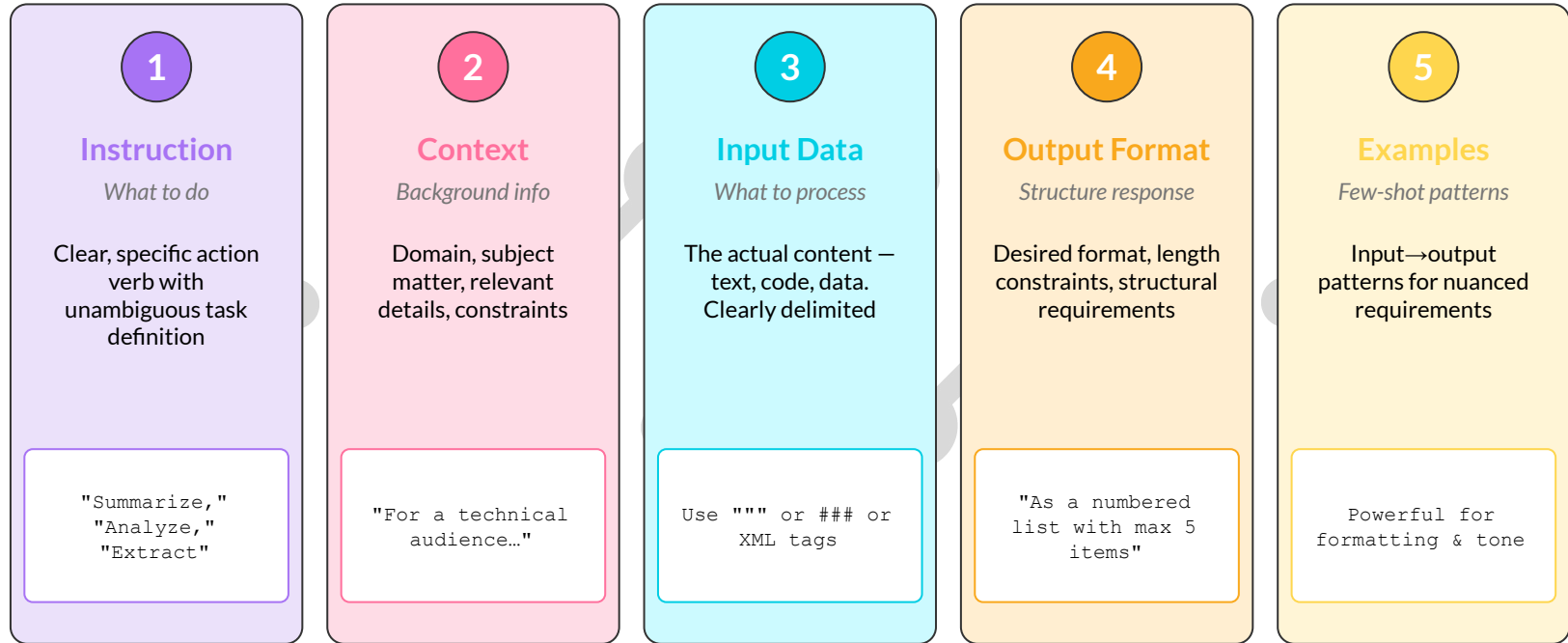
Generative AI & Agentic AI Program



Anatomy of an Effective Prompt



Five Core Components of a Prompt



Prompt Quality Checklist



Specific — Avoid vague instructions — be precise about what you want



Clear — No ambiguous language — one interpretation only



Complete — All necessary context provided — nothing left to guess



Constrained — Boundaries and limitations defined — scope the task



Testable — Can verify if output meets requirements — measurable success



Example: Well-Structured Prompt

INSTRUCTION: Analyze the following customer review sentiment

CONTEXT: E-commerce platform selling electronics
Target: Extract sentiment + key issues + recommendation

INPUT:
"The laptop arrived 2 days late. Battery life is disappointing — only 4 hours vs advertised 8 hours. However, the display quality is excellent and performance is smooth."

OUTPUT FORMAT:

```
{ "sentiment": "mixed",  
  "score": 6,  
  "key_issues": ["late delivery", "battery"],  
  "positives": ["display", "performance"],  
  "recommendation": "..."} }
```

1. Instruction

Clear action verb

2. Context

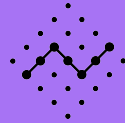
Domain + target

3. Input Data

Delimited content

4. Output Format

Structured JSON



System, User & Assistant Prompts

Message Role Architecture



Modern LLMs use a three-role conversation structure



System Prompt

Sets AI behavior, personality & rules

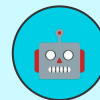
- Set once at conversation start
- Defines overall behavior
- Not visible to end users
- Highest priority



User Prompt

The actual query or task from the human

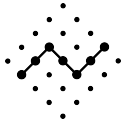
- Changes with each turn
- Contains the specific request
- Can include context & data
- Most dynamic component



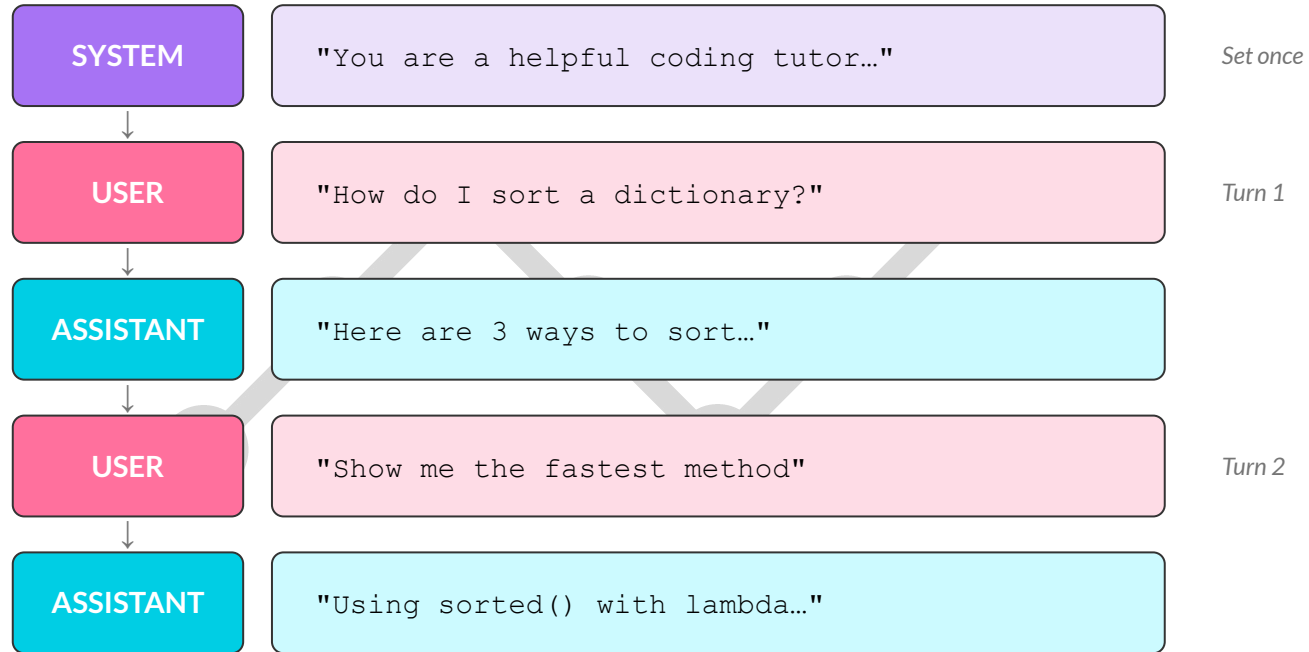
Assistant Response

The AI's generated output

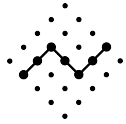
- Follows system guidelines
- Addresses user prompt
- Maintains conversation context
- Coherent and relevant



Conversation Flow Pattern



Key: System prompt is set once. User & Assistant alternate each turn, building context across the conversation.



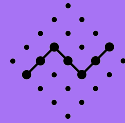
Role Interaction Principles

Aspect	System	User	Assistant
Frequency	Once per session	Every turn	Every turn
Mutability	Static	Dynamic	Dynamic
Purpose	Behavior rules	Task / query	Response
Visibility	User-Created	User-Created	User-Facing
Priority	Highest	Medium	N/A

System: Define persona: "You are a financial analyst..." | Set tone | Establish boundaries

User: Be specific | Provide context | Include constraints | Reference prior turns

Assistant: Follows system rules | Addresses user query | Maintains context across turns



Key Takeaways

Key Takeaways



System prompts are your configuration layer — use them to set guardrails



User prompts should be task-focused and context-rich



Assistant responses inherit behavior from system + react to user



Separation of concerns — don't put system-level instructions in user prompts



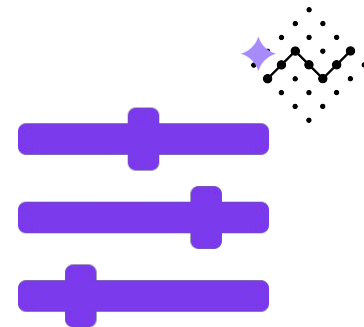
Conversation history — each role builds context across turns

Master these fundamentals — they're the foundation for every advanced prompting technique.

LLM SETTINGS & GENERATION PARAMETERS

NUNNARI ACADEMY

Module 1: Foundations & LLM Essentials | Week 2 | Section 2.1





Getting Oriented

The OpenAI Playground — Your experimentation environment



The OpenAI Playground



System Message

Define the model's behavior, persona, and constraints. Acts as persistent instructions.



User Message

Type the actual user query or task. Supports `{{template variables}}` for dynamic inputs.



Settings Panel

Control generation parameters — Model, Temperature, Top P, Max Tokens, etc.



Tip: Click "New prompt" for each exercise. Run the SAME prompt 3–5 times to observe variation.



Temperature

Controlling randomness and creativity in model output



Temperature Settings

Controls the probability distribution over possible next tokens

Value	Behavior	Output Style	Best For
0	Fully deterministic	Same output every time	Fact extraction, classification, code
0.3	Slightly creative, focused	Minor phrasing variations	Summaries, professional writing, Q&A
0.7	Balanced creativity & coherence	Noticeably different each run	General chatbots, email drafting
1.0	High creativity, may diverge	Unique and diverse each time	Brainstorming, creative writing
1.5–2.0	Very random, often incoherent	Unpredictable, may hallucinate	Experimental/artistic use only

Lower = Deterministic → Higher = Creative



Temperature in Action

Experiment with factual vs. creative tasks

Experiment A: Factual Task

System:

"You are a geography expert."

User:

"What is the capital of Tamil Nadu?"

Temp 0 → Identical every time

Temp 0.7 → Same fact, slightly different wording

Temp 1.5 → May ramble or go off-topic

Experiment B: Creative Task

System:

"You are a creative poet."

User:

"Write a 4-line poem about rain in Coimbatore."

Temp 0 → Boring, identical across runs

Temp 0.7 → Good variety, still coherent

Temp 1.2 → Unexpected metaphors, surprising



Top P

Nucleus Sampling — A different approach to controlling creativity



Top P (Nucleus Sampling)

Defines a probability budget for token selection

Token	Prob.	Cumul.
Chennai	60%	60%
Madurai	20%	80%
Coimbatore	10%	90%
Trichy	5%	95%
Ooty	3%	98%
Kanyakumari	2%	100%

Top P = 0.1

Only the most probable token

Top P = 0.8

Chennai + Madurai in pool

Top P = 0.95

Top 4 tokens are candidates

Top P = 1.0

All tokens — no filtering



Tip: Alter Temperature OR Top P — never both. If adjusting one, keep the other at default (1.0).



Max Tokens

Controlling output length, cost, and response quality



Why Max Tokens Matters

1

Cost Control

Every output token costs money. Capping at 100 tokens costs ~5x less than 500 tokens.

2

Response Quality

Uncapped models tend to ramble. Shorter responses are often better and more focused.

3

Latency

Fewer output tokens = faster response time. Critical for real-time applications.

4

Structure

Forces the model to prioritize important information first and stay concise.



Tip: In production apps, always set `max_tokens` explicitly. For FAQ chatbots, 150–300 tokens is often sufficient.



Max Tokens Comparison

System: "Be direct and informative." — User: "Explain what machine learning is."

Max Tokens	Expected Output	Observation
20	Cuts off mid-sentence — incomplete	Too short to be useful
50	One clean paragraph, core idea	Concise but complete — good for summaries
200	Detailed with examples and types	Comprehensive — good for education
1000	History, applications, comparisons	May over-explain — not always needed



Stop Sequences

Structural control over output format and length



Stop Sequences in Action



Controlling List Length

Stop sequence: "6."
Model generates items 1–5 and stops. No need to say "give me 5" in the prompt.



Stopping at Delimiters

Stop sequence: """"
Prevents extra text after JSON output. Clean, parseable responses.



Single-Answer Extraction

Stop sequences: "\n" and "."
Model outputs just "Positive" — no explanation, no hedging.



Frequency & Presence Penalty

Controlling repetition and encouraging diversity



Frequency vs. Presence Penalty

Both range from 0 to 2.0 — default is 0 (no penalty)

Frequency Penalty

Proportional penalty

A word used 5 times gets 5× the penalty.

Effect: Reduces word-level repetition. Good for avoiding "the" or "is" appearing too often.

Presence Penalty

Flat penalty (binary)

Used 1 time = same penalty as 10 times.

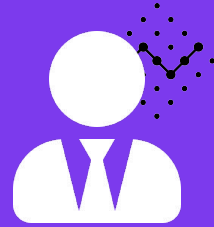
Effect: Encourages topic diversity. Pushes the model to introduce new concepts.



Tip: Alter Frequency Penalty OR Presence Penalty — not both aggressively. Start with 0.2–0.5 and test.

Freq. Penalty → Less repetition

Pres. Penalty → More diversity



System Message

The most powerful control — defining persona and behavior



Same Query, Different Personas

User: "What should I know about investing in mutual funds?"

1

Financial Advisor

"Certified advisor with 20 years of experience. Provide balanced, professional advice. Include risk disclaimers."

Formal tone, structured advice, risk factors, disclaimers

2

Friend Over Coffee

"Friend who is good with money. Explain casually using simple language and relatable examples."

Casual tone, analogies like "potluck", no jargon

3

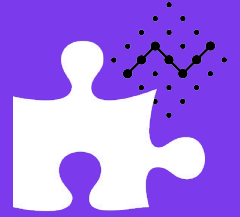
Tamil Finance YouTuber

"Popular Tamil financial YouTuber. Use Tanglish. Examples for middle-class Indian families."

Tanglish, SIP/LIC references, relatable scenarios

Combining Parameters

Real-world recipes for production use cases





Real-World Parameter Recipes

Proven combinations for common use cases

Use Case	Temp	Top P	Max Tok	Freq P	Why It Works
FAQ Chatbot	0	1.0	150	0	Deterministic, concise, consistent
Code Generation	0	0.95	500	0	Precise logic, room for full functions
Creative Writing	0.9	1.0	800	0.3	High creativity, reduced repetition
Email Drafting	0.3	1.0	300	0.2	Professional with natural flow
Data Extraction	0	1.0	100	0	Maximum precision, minimal output
Brainstorming	1.2	1.0	500	0.5	Maximum diversity, no repeats
Classification	0	1.0	10	0	Single label, deterministic, cheap
Translation	0.2	1.0	500	0	Faithful to source, minor variation



Quick Reference Cheat Sheet

Consistent, factual answers

Temperature ↓

0 – 0.2

Creative, varied output

Temperature ↑

0.8 – 1.2

Limit word choices

Top P ↓

0.1 – 0.3

Keep responses short

Max Tokens ↓

50 – 150

Reduce word repetition

Freq. Penalty ↑

0.3 – 0.8

Encourage new topics

Pres. Penalty ↑

0.3 – 0.8



The Two Golden Rules

1

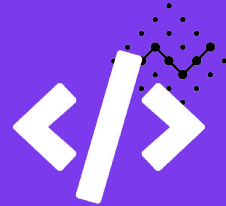
Change Temperature OR Top P — never both.

If adjusting Temperature, keep Top P at 1.0 (default). If adjusting Top P, keep Temperature at 1.0. Changing both creates unpredictable interactions.

2

Change Frequency Penalty OR Presence Penalty — never both aggressively.

Start with small values (0.2–0.5) and test. Aggressive settings on both create unnatural, incoherent output.



From Playground to Code

Mapping Playground settings to the OpenAI API



Playground → API Mapping

Every Playground setting maps directly to an API parameter

Playground Setting	API Parameter	Default
Model dropdown	model	(required)
System message	messages[0] role: "system"	(optional)
Temperature	temperature	1.0
Top P	top_p	1.0
Max Length / Max Tokens	max_tokens	(model limit)
Stop Sequences	stop	None
Frequency Penalty	frequency_penalty	0
Presence Penalty	presence_penalty	0



Context Window Management

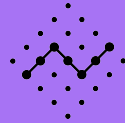


Strategies & Common Pitfalls

Optimizing LLM Performance, Costs & Response Quality

Nunnari Academy

Generative AI & Agentic AI Program • Week 2



Management Strategies

Strategy Comparison (1 of 2)



Strategy	Complexity	Cost Savings	Quality Impact	Best Use Case
Sliding Window	Low	Medium	Medium	Real-time chat
Summarization	Medium	High	Medium	Long conversations
RAG	High	Very High	High	Enterprise apps
Hierarchical Summary	High	High	High	Complex projects
Contextual Compression	Medium	Very High	Medium	Q&A systems

Strategy Comparison (2 of 2)



Strategy	Complexity	Cost Savings	Quality Impact	Best Use Case
Semantic Chunking	Medium	High	High	Document processing
Prompt Caching	Low	Very High	None	High-volume APIs
Intelligent Pruning	Medium	Medium	High	Mixed conversations
External Memory	High	Very High	Medium	User profiles
Multi-Pass	Medium	Low	High	Large documents

Strategy Deep Dive (1 of 2)



1

Sliding Window: Keep only the most recent N messages/tokens, dropping older content when limit is reached

2

Summarization: Periodically condense older conversation history into summaries while keeping recent messages detailed

3

RAG: Store full history externally and retrieve only relevant portions based on current query

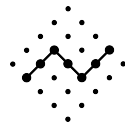
4

Hierarchical Summary: Create multi-level summaries at different granularities (detailed recent, medium mid, high-level old)

5

Contextual Compression: Use an LLM to extract only query-relevant information from the full context before processing

Strategy Deep Dive (2 of 2)



6

Semantic Chunking: Divide content into meaningful topic-based segments and process only relevant chunks

7

Prompt Caching: Cache frequently reused context portions (like system prompts) to reduce token costs on subsequent requests

8

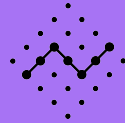
Intelligent Pruning: Selectively remove less important messages based on relevance scoring (recency, importance, engagement)

9

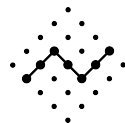
External Memory: Store detailed information in databases and keep only references or IDs in the active context

10

Multi-Pass Processing: Break complex tasks into multiple focused passes, each processing different subsets of the content



Common Pitfalls & How to Avoid



Common Pitfalls (1 of 3)



Vague Instructions — Being too general leads to inconsistent results. Be specific about scope, format, and desired outcomes.



Missing Context — Not providing enough background prevents the model from understanding the task properly.



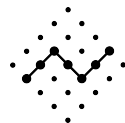
Multiple Questions at Once — Combining unrelated questions leads to incomplete or unfocused responses.



Assuming Real-Time Info — Asking about current events without using appropriate search tools.



No Output Format — Leaving format ambiguous results in outputs that don't match your structural needs.



Common Pitfalls (2 of 3)



Ignoring Token Limits — Including unnecessary content or requesting overly long outputs that waste tokens and increase expenses.



Leading / Biased Language — Phrasing prompts to push toward specific answers rather than requesting objective analysis.



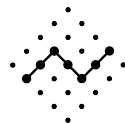
Not Using Examples — Expecting the model to understand complex formats without demonstrations (few-shot learning).



Overcomplicating Simple Tasks — Adding unnecessary complexity or constraints that confuse the model or reduce output quality.



Ignoring System Prompt — Not leveraging system prompts for consistent behavior or contradicting them in user messages.



Common Pitfalls (3 of 3)



Not Iterating — Expecting perfect results on first try without testing variations and improving based on outcomes.



Hallucination Encouragement — Prompting in ways that encourage making up information rather than acknowledging uncertainty.



Neglecting Edge Cases — Not considering how prompts handle unusual inputs, empty data, or boundary conditions.



Wrong Temperature — Using inappropriate generation settings for the task (high temperature for facts, low for creativity).



Forgetting to Validate — Blindly trusting model outputs without verification, testing, or human review.



Core Prompt Engineering Techniques



Five Foundational Methods Every AI Practitioner Needs

Module 1: Foundations & LLM Essentials

Nunnari Academy

Generative AI & Agentic AI Program

Five Techniques at a Glance



Prompt engineering is crafting effective instructions for LLMs to produce desired outputs.

1

Zero-Shot

Direct instructions,
no examples needed

2

Few-Shot

Teach through 1–5
input-output
examples

3

Role-Based

Assign persona or
expertise to the LLM

4

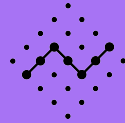
Structured Output

Request JSON,
XML, Markdown
formats

5

Templates

Parameterized,
reusable prompt
patterns



Technique Deep Dives



1 Zero-Shot Prompting

Definition: Getting results from an LLM without providing any examples. Simply give clear, direct instructions and rely on the model's pre-trained knowledge.

When to Use

- Simple, well-defined tasks
- Tasks where the LLM understands the domain
- Quick prototyping and testing

Example

Prompt:

Classify this review as positive or negative:

"The product arrived late but the quality exceeded expectations."

Key Insight: Start here! Zero-shot is your baseline — add complexity only when needed.



2

One-Shot & Few-Shot Prompting

Definition: Teach the LLM through examples before presenting the actual task. Provide 1-5 input-output pairs that demonstrate the desired pattern, format, or style.

When to Use

- Tasks requiring specific formatting
- Domain-specific understanding
- Consistent style or tone
- Complex extraction tasks

Example

Extract name and email from text:

Input: "Contact John at john@email.com"

Output: {"name": "John", "email": "john@..."}

Input: "Reach Sarah via sarah.k@co.com"

Output: {"name": "Sarah", "email": "sarah..."}

Input: "Email Mike at mike.j@corp.org"

Output: ?

Key Insight: Examples teach pattern + format + style all at once. 2-3 examples usually suffice.



3

Role-Based Prompting

Definition: Assign a specific persona, expertise, or perspective to the LLM. This helps the model respond with appropriate context, knowledge depth, and communication style.

When to Use

- Domain expertise requirements
- Specific perspective or viewpoint
- Particular communication style
- Professional or technical contexts

Example

Prompt:

You are a **senior Python developer**.

Review this code and suggest improvements:

```
def calc(a,b): return a+b
```

Key Insight: Roles shape tone, depth, and perspective. Combine with system prompts for maximum effect.



4

Structured Output Generation

Definition: Request responses in specific formats like JSON, XML, or Markdown. Makes output machine-readable and easy to integrate with other systems.

When to Use

- API integrations
- Database operations
- Data pipeline processing
- Automated workflows

Example

Prompt:

Extract product details in JSON:

"The iPhone 15 Pro costs \$999 and comes in 128GB, 256GB."

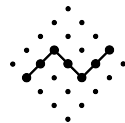
Return format:

```
{
  "product": "",
  "price": "",
  "variants": []
}
```

Key Insight: Always provide the exact schema you want — LLMs follow templates precisely.

5

Prompt Templates & Reusability



Definition: Parameterized prompts that can be reused with different inputs. Maintain consistency and scale prompt engineering across multiple use cases.

When to Use

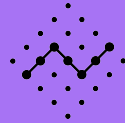
- Repetitive tasks with varying inputs
- Maintaining consistency across apps
- Scaling prompt engineering efforts
- Version control and testing

Example

```
template = """
Translate the following
{source_language} text to {target_language}:
Text:
{input_text}
"""

# Usage
prompt = template.format(
    source_language="English",
    target_language="Tamil",
    input_text="Hello, how are you?"
)
```

Key Insight: Templates turn one-off prompts into scalable, version-controlled assets.



Summary & Best Practices

Technique Summary



Technique	When to Use	Key Benefit
Zero-Shot	Simple, well-defined tasks	Quick and direct
Few-Shot	Tasks needing specific format or style	Learn from examples
Role-Based	Domain expertise needed	Contextual expertise
Structured Output	System integrations	Machine-readable format
Templates	Repetitive tasks	Reusable and scalable

Best Practices



Start simple — begin with zero-shot, add complexity only when needed



Be specific — clear instructions yield better results than vague requests



Test and iterate — experiment with different techniques to find what works best



Combine techniques — use multiple approaches together (e.g. role-based + structured)



Document your prompts — create a library of templates for future reuse

These five techniques form the foundation of professional prompt engineering.



Advanced Prompt Engineering Strategies



From Chain-of-Thought to Meta-Prompting

Module 1: Foundations & LLM Essentials • Section 2.3

Nunnari Academy

Generative AI & Agentic AI Program

Six Advanced Strategies at a Glance



1

Chain of Thought

Step-by-step reasoning for complex problems

2

Tree of Thought

Explore multiple reasoning paths in parallel

3

Self-Consistency

Multiple runs, pick the most consistent answer

4

ReAct Pattern

Interleave reasoning with tool use actions

5

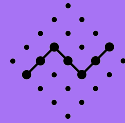
Prompt Chaining

Sequential sub-prompts, output feeds next

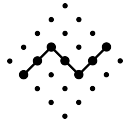
6

Meta-Prompting

Use LLMs to generate & optimize prompts



Strategy Deep Dives



1

Chain of Thought (CoT) Prompting

Concept: Explicitly instruct the LLM to show its reasoning step-by-step before arriving at a final answer.

Example: Basic CoT

Q: A store has 15 apples. They sell 40% in the morning and 6 more in the afternoon. How many remain?

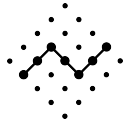
Prompt: "Let's think step by step"

1. Starting apples: 15
2. Morning sales: $15 \times 0.40 = 6$
3. After morning: $15 - 6 = 9$
4. Afternoon sales: 6
5. Final remaining: $9 - 6 = 3$ apples

Benefits

- ✓ Reduces calculation errors
- ✓ Makes debugging easier
- ✓ Improves math, logic & reasoning
- ✓ Enables verification of steps

Breaking problems into steps
improves accuracy & transparency



2

Tree of Thought (ToT)

Concept: Explore multiple reasoning paths simultaneously, evaluating each branch before selecting the best solution.

CoT vs ToT

CoT: One linear path
Step 1 → Step 2 → Step 3 → Answer

ToT: Multiple parallel paths
Problem → Approach 1 (Pros/Cons)
→ Approach 2 (Pros/Cons)
→ Approach 3 (Pros/Cons)
→ **Evaluate** → **Best approach**

Use Cases

- Strategic planning
- Complex problem-solving with multiple valid paths
- Creative writing with multiple plot directions
- Game playing and decision trees



3

Self-Consistency & Ensemble

Self-Consistency

Generate multiple independent reasoning paths and select the most consistent answer.

1. Run same problem 5x with different phrasings

2. Compare all answers

3. Select majority answer or synthesize insights

Ensemble Prompting

Combine responses from different prompt strategies or models.

Response 1: CoT reasoning

Response 2: Role-based expert

Response 3: Data-driven stats

→ Synthesize all perspectives

When to Use

- ✓ High-stakes decisions
- ✓ Ambiguous problems
- ✓ Validation of critical outputs
- ✓ Reducing hallucination risk

Trade-off: Higher cost and latency due to multiple API calls — use when accuracy justifies the expense.



4 ReAct (Reasoning + Acting)

Concept: Interleave reasoning (thought) and action (tool use) in an iterative loop.

The ReAct Loop

1 **Thought:** Analyze current situation

2 **Action:** Execute a tool or step

3 **Observation:** Process the result

4 **Repeat:** Until task is complete

Example

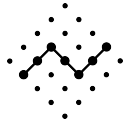
Q: What is the GDP of the country where the Eiffel Tower is?

Thought: Need to identify the country
Action: Search "Eiffel Tower location"
Obs: It's in Paris, France

Thought: Now I need France's GDP
Action: Search "France GDP 2024"
Obs: ≈ \$3.05 trillion

Answer: France's GDP ≈ \$3.05 trillion

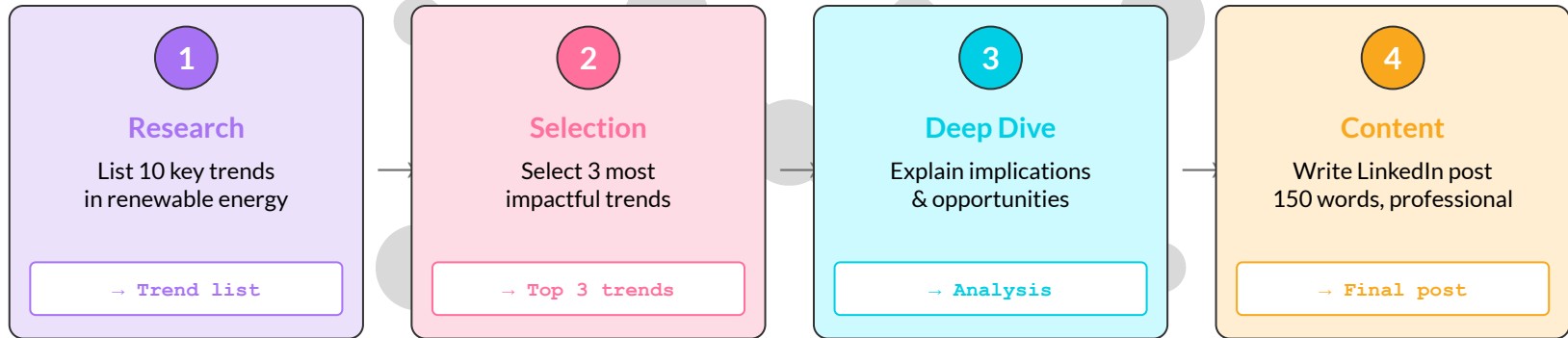
Foundation for AI Agents: Grounds reasoning in real data via tool use • Reduces hallucination • Makes behavior transparent



5 Prompt Chaining

Concept: Break complex tasks into sequential sub-prompts, where each prompt's output feeds into the next.

Example: Content Creation Pipeline



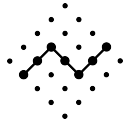
Implementation Patterns

Sequential: Linear flow
(1→2→3)

Conditional: Branches on
results

Parallel: Multiple chains at
once

Recursive: Outputs loop back



6 Meta-Prompting

Concept: Use LLMs to generate, optimize, or refine prompts themselves — prompts that create prompts.

1

Prompt Generation

Create 5 prompt variations for a task, each using different techniques, optimized for accuracy and conciseness.

2

Prompt Optimization

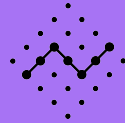
Take a basic prompt like "Summarize this article" and improve specificity, format, key aspects, and constraints.

3

Prompt Debugging

Diagnose why a prompt produces verbose, incomplete, or inconsistent outputs, then generate an improved version.

Use Cases: Automated prompt libraries • A/B testing variations • Cross-model adaptation • Domain-specific templates



Summary & Best Practices

Techniques Summary



Strategy	Core Idea	Best For	Key Benefit	Trade-off
Chain of Thought	Step-by-step reasoning	Math, logic, analysis	Transparent reasoning	Longer outputs
Tree of Thought	Multiple parallel paths	Strategic planning	Explores alternatives	Higher complexity
Self-Consistency	Multiple runs, majority vote	High-stakes decisions	Reduces hallucination	Multiple API calls
ReAct Pattern	Reasoning + tool use loop	Data-grounded tasks	Real data via tools	Requires tool setup
Prompt Chaining	Sequential sub-prompts	Complex pipelines	Modular & debuggable	Multi-step latency
Meta-Prompting	LLMs optimize prompts	Prompt libraries & A/B	Scalable automation	Extra LLM call

Each strategy excels in different contexts — combine them for maximum effectiveness.

Best Practices



Start simple — add complexity only when needed



Document strategies — track what works and why



Measure effectiveness — use clear metrics to evaluate



Iterate on real outputs — refine based on actual results



Balance sophistication — weigh quality gains vs cost/latency

Advanced strategies transform LLMs from simple text generators into sophisticated reasoning systems capable of complex, multi-step problem solving.



Prompt Engineering Best Practices



From Iteration to Production-Ready Prompts

Module 1: Foundations & LLM Essentials • Section 2.4

Nunnari Academy

Generative AI & Agentic AI Program

Five Best Practice Areas



Production-ready prompts require the same rigor as software engineering.

1

Iterative Refinement

Systematic
test-measure-refine
cycles for reliable
prompts

2

A/B Testing

Data-driven
comparison of prompt
variations

3

Handling Hallucinations

Grounding outputs in
facts and preventing
fabrication

4

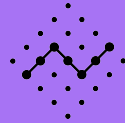
Bias Detection

Identifying and
mitigating systematic
unfairness

5

Docs & Versioning

Treating prompts as
code with proper
version control



Iterative Prompt Refinement



1 The Refinement Cycle

Principle: Effective prompts are rarely perfect on the first try. Systematic refinement produces better, more reliable results.

1

Define Success Criteria

2

Create Initial Prompt

3

Test & Evaluate

4

Identify Failure Patterns

5

Refine & Iterate

6

Repeat Until Satisfactory

Step 1: Success Criteria

✓ **Accuracy:** Factually correct outputs

✓ **Consistency:** Similar quality across runs

✓ **Completeness:** Addresses all aspects

✓ **Format:** Follows specified structure

✓ **Tone:** Matches desired voice

✓ **Efficiency:** Uses tokens optimally

Start Simple, Add Complexity



Add one improvement at a time to isolate what works.

Baseline

(Too vague)

Summarize this
feedback.

✗ No format, no length,
no structure specified

Iteration 1

(Add structure)

Summarize this customer
feedback. Provide:
- 3-5 key themes
- Supporting examples
for each

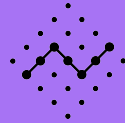
✓ Structure added, but
length still open-ended

Iteration 2

(Add constraints)

Summarize this customer
feedback in 150-200 words.
Provide:
- 3-5 key themes
- 1-2 examples per theme
- Use neutral, objective
tone

✓ Length + tone + format
all constrained



A/B Testing Prompts



2

A/B Testing Prompts

Goal: Systematically compare prompt variations to identify the most effective approach. Data over intuition.

When to A/B Test

- ✓ Multiple valid approaches exist
- ✓ Small changes might have large impacts
- ✓ Optimizing for specific metrics
- ✓ Scaling to production

Skip When

- ✗ Prompt clearly fails or succeeds
- ✗ Time/resource constraints are tight
- ✗ Use case is one-time or exploratory

Example Results

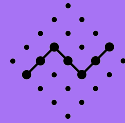
Variant	Accuracy	Avg Tokens	Cost/Req	Rating
A (Control)	72%	450	\$0.023	3.2/5
B (Structured)	85%	520	\$0.026	4.1/5
C (Role-based)	88%	480	\$0.024	4.5/5

🏆 Winner: Variant C — Best accuracy + user rating, reasonable cost

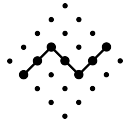
Hypothesis: Adding "think step-by-step" will improve accuracy by 20%
100 problems/variant

Metric: % correct answers

Sample:



Handling Hallucinations



3

Handling Hallucinations

Hallucination: When LLMs generate plausible-sounding but factually incorrect or fabricated information.

1

Factual Errors

Incorrect dates, names,
numbers

Non-existent events
or people

Wrong relationships
or causality

2

Fabricated References

Fake citations or
sources

Made-up research
papers

Invented statistics

3

Logical Inconsistencies

Contradictions within
response

Impossible scenarios

Flawed reasoning
chains



Hallucination Prevention Strategies

Ground in Context

✗ Encourages hallucination

```
"What are the latest
developments in
quantum computing?"
```

✓ Prevents hallucination

```
"Based on {article text},
summarize developments.
Only include info
explicitly stated."
```

Explicit Uncertainty

✗ Encourages hallucination

```
"Answer this question
about current events"
```

✓ Prevents hallucination

```
"If uncertain:
- State confidence level
- Explain what is missing
- Never fabricate details"
```

Request Citations

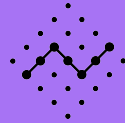
✗ Encourages hallucination

```
"Tell me facts about
this topic"
```

✓ Prevents hallucination

```
"For each claim:
- Provide the source
- Acknowledge cutoff
- State if uncertain"
```

Also: Use RAG for factual domains • Lower temperature (0.1–0.3) • Test with adversarial examples • Implement verification steps



Bias Detection & Mitigation



4

Bias Detection & Mitigation

Bias: Systematic patterns in LLM outputs that unfairly favor or discriminate against certain groups, perspectives, or outcomes.

Types of Bias

Social Bias

Gender stereotypes, racial/ethnic prejudice, age discrimination, cultural assumptions

Selection Bias

Overrepresenting certain viewpoints, ignoring minority perspectives, geographic centrism

Representation Bias

Assuming defaults (e.g. doctor = male), stereotypical associations

Mitigation Strategies

Strategy 1: Explicit Fairness

Generate interview questions.

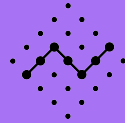
Requirements:

- Role-relevant only
- Avoid: age, gender, race
- Focus: skills & problem-solving

Strategy 2: Balanced Representation

Generate 6 success stories with:

- 3 women, 3 men
- Variety of industries
- Different career stages
- International perspectives



Documentation & Version Control

5

Documentation & Version Control



Why? Prompts are critical business logic that evolve, require debugging, need collaboration, must be reproducible, and have performance implications.

Essential Documentation

Purpose: What does this prompt do?

Version: Current version + date

Model Config: Model, temperature, tokens

Input Format: Expected input structure

Success Criteria: How to evaluate quality

Known Limits: What doesn't work well

Changelog: Version history

Maintainer: Who owns this prompt

Semantic Versioning

MAJOR . **MINOR** . **PATCH**

MAJOR: Breaking changes (new output format)

v1.1.0 → v2.0.0: *Changed to JSON*

MINOR: Backward-compatible improvements

v1.0.1 → v1.1.0: *Added examples*

PATCH: Bug fixes, small tweaks

v1.0.0 → v1.0.1: *Fixed typo*

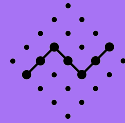
Team Practices

✓ Code review
for prompts

✓ Change log
discipline

✓ Performance
monitoring

✓ Rollback plan



Summary



The Path to Production-Ready Prompts

- 1 **Iterate systematically** — Test, measure, refine
- 2 **A/B test rigorously** — Data over intuition
- 3 **Minimize hallucinations** — Ground in facts, verify claims
- 4 **Audit for bias** — Test diverse perspectives
- 5 **Document thoroughly** — Treat prompts as code

Prompt engineering is software engineering. Apply the same rigor: version control, testing, documentation, and continuous improvement.



Thank You!



Nunnari Academy
Generative AI & Agentic AI Program